Arquitectura de Saneamiento de Mazos

https://gemini.google.com/app/503a0364fa94a2d4

User prompt: Necesito definir claramente la forma oficial para sanear tarjetas de un mazo, comprobar la calidad absoluta de dicha función de "sanear cluster" (que se supone es la ultima forma propuesta para sanear un mazo) y si es la mejor forma de abordar el problema siguiendo estos objetivos (A continuación leeras muchas cosas que el sistema ya hace y otras que no, además como comente antes no existe un caso de uso formal para sanear tarjetas ya que fuimos cambiándolo con el tiempo y actualmente hay código muerto y nada concreto, ya que todo comenzó como un simple creador de cards, por ejemplo el "Explorar y Seleccionar Tarjeta de anki", el "sanear cluster" etc, deberás investigar el código y hacer las respectivas preguntas antes de siquiera empezar a hacer modificaciones al sistema): Objetivos: - Transformar mi card monolítica hecha a mano con mucho texto o imágenes a una o unas de calidad según patrones de neurociencia y ciencia del aprendizaje. - Dicha transformación debe ser inteligente, ya que existirán cards que toquen mas de un tema y en algún punto el sistema podría sugerir cards parecidas, o similares desde el punto pedagógico (ya sea con las nuevas cards del lote propuestas o con cards existentes, cada card debe tener una justificación. - Dichas cards existentes podrían ser de mala calidad, por lo cual el sistema deberá notificarlo, y darle su turno de procesamiento (con eso evitamos que el sistema asuma negativamente). - el saneamiento del mazo no debe ser aislado (por eso por ejemplo se propuso un saneamiento por clusters pero quiero cuestionar absolutamente todo sobre el código actual para definir si es la forma mas profesional para solucionar el problema). - Después del saneamiento que el sistema tenga el conocimiento del porcentaje de saneamiento del mazo, la calidad y el nivel de abordaje de los temas (Ejemplo: "es suficientes estas cards para que el usuario entienda el modelo OSI? o existen brechas? considerando que el usuario es totalmente principiante). - El sistema sabra los temas abordados con el objetivo de detectar temas abordados, brechas, y futuros temas para ir expandiendo progresivamente el mazo. Tal vez se me escapen mas consideraciones o cosas que quiero, eres totalmente libre de proponerlos, tu tarea será entender cuales están actualmente en el sistema y cuales necesitan ajustes. Con esto espero: que el sistema sepa que dicho mazo tiene un objetivo concreto y que debe acoplarse a el. Que me ayude a sanear mis mazos siguiendo patrones de ciencia del aprendizaje, que las x cards creadas para cada tema me permitan entender y retener el conocimiento en su totalidad un tema (considerando que soy inexperto). y que al finalizar se tenga la certeza de que no hay brechas, el mazo es de calidad y que es seguro empezar a expandir el mazo a nuevos temas para asi llevar al usuario a una experticia total en el tema. Ademas que dicho objetivo global sea dinamico o que tenga objetivos secundarios (EJ: "supongamos que el usuario ya sabe de 0 a 100 los fundamentos de redes, y quiere conocer mas de ámbitos cloud como AWS, Cisco, Oracle etc, y que desea certificarse en algo). (actualmente se llama plan de estudios (Fase 2))

Response: 1. Estado de Lectura del Archivo Adjunto y Topología Detectada

El archivo comprimido (.zip) fue procesado por el lector de adjuntos como un flujo binario (deflate), lo que permite ver **toda la estructura de directorios y nombres de módulos** a través de las cabeceras del archivo, pero impide leer el texto interno de cada script .py y .tsx.

Sin embargo, el árbol de archivos expuesto ya evidencia por qué sientes que hay código muerto y flujos

solapados:

- **Fragmentación en el Frontend (frontend/src/components/)**: Coexisten múltiples puntos de entrada para manipular o auditar tarjetas existentes: AnkiCardPickerModal.tsx, RefactorNoteModal.tsx, ClusterSanitationModal.tsx, CognitiveAuditModal.tsx, CurationQueueBanner.tsx y DeckStudioModal.tsx.
- **Fragmentación de Servicios en el Backend (backend/app/services/ y core/)**: El sistema separa el agrupamiento vectorial (clustering.py) y el saneamiento (sanitation_service.py) de la inteligencia del mazo (deck_intelligence_service.py), la topología (deck_dag_service.py, ontology_extractor_service.py), la expansión (deck_expansion_service.py), el perfilado de objetivos (deck_interview_service.py, exam_blueprint_service.py) y las cinco capas de auditoría (cognitive_auditor_service.py, pedagogical_discriminator_service.py, pedagogical_judge_service.py, semantic_auditor_agent.py, socratic_red_teaming_service.py).
- **Documentación de Auditoría Previa (docs/)**: Ya existe incluso un documento específico llamado docs/SANITATION_WORKFLOW_AUDIT.md junto con docs/AUDIT_FINDINGS.md, docs/ARCHITECTURE.md y docs/ROADMAP.md.

Para auditar línea por línea la implementación actual sin adivinar nada, **adjunta directamente como archivos sueltos (o concatenados en un único .txt / .md) estos archivos clave**:

1. docs/SANITATION_WORKFLOW_AUDIT.md, docs/AUDIT_FINDINGS.md y docs/ROADMAP.md.
2. backend/app/services/sanitation_service.py, backend/app/services/deck_intelligence_service.py y backend/app/core/clustering.py.
3. backend/app/services/deck_dag_service.py, backend/app/services/deck_expansion_service.py y backend/app/services/deck_interview_service.py.
4. backend/app/api/routes.py y backend/app/domain/models.py.
5. frontend/src/components/ClusterSanitationModal.tsx, frontend/src/components/DeckStudioModal.tsx y frontend/src/App.tsx.

2. Cuestionamiento Arquitectónico: ¿Es "Sanear Cluster" la mejor forma de abordar el problema?

Si evaluamos tus objetivos desde ingeniería de software y ciencia cognitiva, **el "Saneamiento por Cluster" (agrupación por similitud de embeddings) NO debe ser el caso de uso raíz ni el director del flujo, sino un mecanismo auxiliar dentro de un pipeline gobernado por el Grafo de Precedencia (DAG) y el Plan de Estudios**.

A continuación te detallo por qué el enfoque centrado exclusivamente en *clusters* choca con tus objetivos y cómo debería reestructurarse conceptualmente:

Objetivo definido por ti	Limitación del enfoque puro de "Sanear Cluster"	Arquitectura Profesional Propuesta (DAG-First + Cluster-Assisted)
1. Fisión de card monolítica (texto denso o imágenes) en múltiples	Un *cluster* de embeddings agrupa por cercanía léxica/semántica promedio.	**Paso 1 (Deconstrucción Ontológica previa al Clustering):** Antes de

Objetivo definido por ti	Limitación del enfoque puro de "Sanear Cluster"	Arquitectura Profesional Propuesta (DAG-First + Cluster-Assisted)
cards atómicas	Una tarjeta monolítica que toca 4 subtemas distintos tiene un vector "promedio" ruidoso que cae en un único cluster artificial o queda como *outlier*.	agrupar, toda tarjeta monolítica debe pasar por una extracción de proposiciones atómicas y conceptos (ontology_extractor_service.py / inspect_material_service.py). El clustering se hace sobre las **unidades conceptuales extraídas**, no sobre la nota monolítica cruda.
2. Detección inteligente de multi-tema, sugerencia de cards similares (lote nuevo vs. existentes) y justificación individual	Si procesas un cluster cerrado, el LLM solo "ve" las tarjetas que cayeron dentro de ese radio vectorial. Si una tarjeta monolítica se divide en los temas *A*, *B* y *C*, el tema *C* puede solaparse con una tarjeta existente que quedó en *otro* cluster.	**Búsqueda Vectorial Cruzada post-Fisión (Cross-Deck Retrieval):** Cuando la nota monolítica se divide en *N* borradores candidatos, cada borrador ejecuta una consulta de similitud contra el almacén del mazo (deck_knowledge_store.py) y el lote actual para adjuntar: (a) justificación pedagógica, (b) vínculo al nodo del tema, y (c) alerta de colisión/similitud con notas existentes fuera del cluster original.
3. No asumir negativamente sobre cards existentes de mala calidad (notificar y encolar turno)	En un saneamiento de cluster convencional, se suele intentar "fusionar o reemplazar todo el cluster de golpe" o descartar las existentes sin auditarlas formalmente.	Máquina de Estados por Tarjeta en el Almacén (deck_knowledge_store.py): Cada nota existente en Anki debe tener un estado auditable (UNAUDITED, FLAGGED_LOW_QUALITY, QUEUED_FOR_SANITATION, SANITIZED_VERIFIED). Si al sanear la Nota *X* aparece la Nota *Y* como similar pero defectuosa (según

Objetivo definido por ti	Limitación del enfoque puro de "Sanear Cluster"	Arquitectura Profesional Propuesta (DAG-First + Cluster-Assisted)
		cognitive_auditor_service.py / pedagogical_discriminator_service.py), el sistema **no la borra a ciegas**: la marca como QUEUED_FOR_SANITATION (o permite sumarla al lote actual si el usuario lo aprueba) para que tenga su propio turno de fisión/refactorización.
4. Saneamiento no aislado	Sanear un cluster aislado sin saber qué prerrequisitos ya están cubiertos en el mazo rompe la coherencia pedagógica (puedes crear una tarjeta que usa términos aún no definidos en nodos previos).	Saneamiento en Orden Topológico (deck_dag_service.py): El saneamiento del mazo debe seguir el orden de dependencias del DAG (de fundamentos a conceptos compuestos). Los clusters sirven para resolver redundancias **dentro** de un mismo nodo temático o entre nodos vecinos.
5. Métricas reales post-saneamiento (% de saneamiento, calidad y suficiencia para un principiante)	El porcentaje de "clusters saneados" es una métrica de vanidad: puedes tener el 100% de las tarjetas actuales saneadas con excelente formato Cloze y aun así cubrir solo el 20% del Modelo OSI (dejando brechas críticas para un principiante).	**Doble Métrica Separada:** 1. **Salud Cognitiva del Mazo Actual (% Saneado):** Total Tarjetas en AnkiTarjetas Verificadas ▪ 2. **Cobertura Pedagógica del Tema (% Suficiencia):** Comparación entre los conceptos requeridos para dominar el nodo (ej. Modelo OSI nivel principiante) y los conceptos efectivamente

Objetivo definido por ti	Limitación del enfoque puro de "Sanear Cluster"	Arquitectura Profesional Propuesta (DAG-First + Cluster-Assisted)
		cubiertos por tarjetas verificadas (ThematicCoverageChecklist.tsx / test_thematic_coverage_audit.py).
6. Objetivo global dinámico y sub-objetivos (ej. Redes 0 a 100 → AWS / Cisco / OCI)	El clustering solo mira lo que *ya existe* en el mazo; es ciego hacia dónde quiere ir el usuario.	Acoplamiento con Plan de Estudios (deck_interview_service.py + exam_blueprint_service.py + deck_expansion_service.py): El mazo tiene un *Perfil de Objetivo Activo* (nivel actual del usuario + meta principal + metas secundarias/blueprints). Ese perfil dicta qué tan exhaustiva debe ser la descomposición al sanear y qué nodos "fantasma" (brechas o temas futuros) se proyectan en el DAG.

3. Preguntas Clave de Diseño (Para responder antes de modificar el sistema)

Mientras subes los archivos de texto para revisar el código actual, necesito que definamos estas decisiones de diseño para que el **Caso de Uso Oficial de Saneamiento** quede blindado y podamos limpiar todo el código muerto:

1. **Punto de entrada único vs. Modales actuales**: Actualmente tienes en el frontend AnkiCardPickerModal, RefactorNoteModal, ClusterSanitationModal, CognitiveAuditModal y DeckStudioModal. ¿Quieres que **todo** el flujo de saneamiento viva unificado dentro de DeckStudioModal (guiado por el mapa del mazo/clusters/cola de curación) y que eliminemos o absorbamos los flujos sueltos como "Explorar y Seleccionar Tarjeta individual" (AnkiCardPickerModal / RefactorNoteModal), o deseas conservar la opción de sanear una tarjeta puntual manualmente pero haciendo que por debajo ejecute exactamente el mismo pipeline contextualizado del mazo?
2. **Comportamiento exacto cuando una tarjeta existente relacionada es de "mala calidad"**: Cuando estás saneando la tarjeta *A* y el sistema detecta que la tarjeta existente *B* toca uno de los mismos subtemas pero viola reglas cognitivas, ¿prefieres que el sistema:
    - **(Opción A - Fusión en caliente opcional):** Te muestre la tarjeta *B* en la misma pantalla con su diagnóstico de mala calidad y un botón *"Absorber en este lote de saneamiento"* (para que el LLM use el contenido de *A*+*B* juntos y reemplace ambas de una vez), o *"Enviar a la cola de turno"* si prefieres no mezclarla ahora.

    - **(Opción B - Estrictamente diferido):** Que solo te avise de la existencia de *B*, la ponga automáticamente en la cola de saneamiento (CurationQueueBanner) con prioridad alta, y cuando llegue el turno de *B* recuerde que ya creaste tarjetas sobre ese tema en el turno de *A* para no duplicarlas.
3. **Destrucción/Archivado de la tarjeta monolítica original en Anki**: Cuando una tarjeta monolítica (1 nota en Anki) se transforma en, digamos, 4 tarjetas atómicas nuevas: ¿qué debe ocurrir en Anki con la nota original para no perder tu historial o programación si aplica? (Ej.: eliminarla definitivamente vía anki_connect_adapter.py, suspenderla y etiquetarla como sanitized_archived, o sobrescribir la nota original con la primera card atómica e insertar las 3 restantes como nuevas).
4. **Criterio de "Certeza de Cero Brechas" antes de Expandir**: Para que el sistema declare un tema como *"100% comprendido sin brechas para un principiante"* y habilite la expansión hacia nuevos temas (ej. pasar de Fundamentos de Redes / OSI a AWS u OCI): ¿quieres que esa validación sea **automática por cobertura ontológica** (el LLM genera el árbol de micro-conceptos obligatorios de "Modelo OSI para principiantes" y verifica que cada micro-concepto tenga al menos 1 tarjeta verificada), o quieres incluir además el paso interactivo de socratic_red_teaming_service.py como prueba de fuego?
5. **Estructura de los Objetivos Dinámicos (Plan de Estudios / Fase 2)**: ¿Cómo quieres modelar la relación entre el objetivo base y los objetivos secundarios en un mismo mazo? Por ejemplo: ¿un mazo tiene una **Ruta Evolutiva por Etapas** (Etapa 1: *Fundamentos de Redes [Activa]* → Etapa 2: *Certificación Cisco/AWS/OCI [En espera o Activa en paralelo]*), de modo que al terminar de sanear y cubrir brechas de la Etapa 1 el mismo mazo desbloquee los nodos de la Etapa 2 en el DAG sin perder las métricas de la Etapa 1?

Comparte tus respuestas a estos 5 puntos junto con el contenido en texto de los archivos listados en la sección 1, y trazaremos la radiografía exacta de qué sirve, qué es código muerto y cómo quedará la arquitectura definitiva.

User prompt: Preguntas Clave: 1. Quiero un lugar unificado para sanear el mazo, la función de tarjeta individual tiene la particularidad que usando el botón "Analizar y planificar", se detiene a analizar antes de crear tarjetas (buscar prerrequisitos, encontrar ejes temáticos, definir numero de cards por eje etc. actualmente es una función muy buena (considera si se puede mejorar con el objetivo de tener dicha consistencia pedagojica en el mazo, porque actualmente desconozco si la hace), pero solo funciona para cards individuales y no permite sanear en un orden lógico (primero conceptos iniciales y luego ir escalando. la otra función dentro de deck studio (sanear cluster), parece apuntar a poner una barra de progreso para sanear pero creo que tampoco tiene una lógica organizacional para ordenar primero las esenciales y creo que tampoco tiene la capacidad de "analizar y planificar" a diferencia de la otra función, además de vivir en un modal dentro de deck studios y no en la pantalla principal. me gustaria un lugar que unifique las virtudes de ambos en la ventana principal (además me gustaria que te aseguraras que dicho espacio tuviera la consistencia que buscamos y no un saneamiento desordenado. (estoy abierto totalmente a modificaciones que aseguren la calidad del saneamiento). 2. Definitivamente opción B, con eso me aseguro que dicha card de mala calidad tenga su espacio de procesamiento propio y asegurar su calidad. 3. eliminarla totalmente, no me interesa el historial ni mantenerla. 4. me gastaría incluir el paso interactivo. 5. me gusta la estructura de objetivos dinámicos que propones.

Response: 1. Nota Técnica sobre el Archivo Subido (KEY_FILES_CONCATENATED.md)

Veo que preparaste el archivo KEY_FILES_CONCATENATED.md con todos los módulos concatenados, pero al subirlo dentro de un contenedor comprimido (.zip), el lector del sistema recibe el binario comprimido

(deflate) y solo deja leer los nombres de los archivos en las cabeceras (PK), ocultando el texto fuente interno.

- **Cómo solucionarlo en 5 segundos:** Sube el archivo **KEY_FILES_CONCATENATED.md** (o renombrado a .txt) **directamente suelto en el chat, sin comprimirlo en .zip**. Al ser texto plano directo, podré leer el 100% de las líneas de código de una sola vez.

2. Definición Oficial del Caso de Uso: "Saneamiento Guiado en el Cockpit Principal"

Con tus 5 respuestas, el problema arquitectónico queda completamente claro: hoy tienes **dos mitades desconectadas**:

1. **La mitad con cerebro pedagógico (MaterialInspectionCockpit.tsx + inspect_material_service.py):** Tiene el flujo de *"Analizar y planificar"* (detecta prerrequisitos, extrae ejes temáticos y calcula cuántas tarjetas requiere cada eje), pero **es ciega al orden del mazo** y opera sobre una tarjeta suelta.
2. **La mitad con visión de lote (ClusterSanitationModal.tsx dentro de DeckStudioModal.tsx + sanitation_service.py):** Tiene barra de progreso de saneamiento, pero **vive encerrada en un doble modal**, agrupa por *clusters* sin orden topológico (no prioriza fundamentos antes que conceptos avanzados) y se salta el paso de *"Analizar y planificar"*.

A continuación se define la **Arquitectura Oficial Unificada** para absorber ambos mundos en la **ventana principal (App.tsx)** y eliminar el código muerto:

Fase 0: Alineación con el Objetivo Dinámico del Mazo (Etapas Evolutivas)

Antes de ordenar o sanear, el mazo carga su **Plan de Estudios Activo** (deck_interview_service.py + exam_blueprint_service.py + ActiveStudyPlanBanner.tsx):

- **Etapa Primaria (ej. *Fundamentos de Redes / Modelo OSI - Nivel Principiante*):** Define el estándar de granularidad (si eres principiante en ese dominio, ningún término técnico se da por sentado; todo requiere definición atómica).
- **Etapas Secundarias / Futuras (ej. *AWS / Cisco / OCI*):** Quedan registradas en la hoja de ruta del mazo para que el sistema sepa hacia dónde dirigir la expansión una vez que la Etapa Primaria alcance 0% de brechas.

Fase 1: Indexación, Auditoría Silenciosa y Cola Topológica (El Fin del "Cluster" Ciego)

Cuando seleccionas un mazo para sanear en la pantalla principal, el sistema **no** te mete en un modal de clusters aleatorios:

1. **Auditoría Inicial y Extracción de Nodos (deck_intelligence_service.py + deck_dag_service.py):** El sistema escanea las notas existentes en Anki, identifica qué temas toca cada tarjeta y evalúa su salud cognitiva (cognitive_auditor_service.py).
2. **Ordenamiento Topológico (De Fundamentos a Complejos):** Usando el DAG (CognitivePrecedenceDAG.tsx / deck_dag_service.py), el sistema construye la **Cola Oficial de Saneamiento (CurationQueueBanner.tsx)**:
    - **Prioridad 1 (Raíces del DAG):** Tarjetas defectuosas o monolíticas que contienen los conceptos más fundamentales (sin prerrequisitos previos).

- **Prioridad 2 (Nodos intermedios):** Tarjetas cuyos conceptos dependen de las raíces ya saneadas.
- **Prioridad 3 (Hojas / Casos avanzados):** Tarjetas de detalles específicos o integración.

Fase 2: Ejecución Turno a Turno en la Ventana Principal ("Analizar y Planificar" Contextualizado)

Al iniciar o continuar el saneamiento desde la barra de progreso en la ventana principal (App.tsx), el sistema carga la **Tarjeta #1 de la Cola Topológica** directamente en el área de trabajo principal:

1. **Inspección y Planificación Consciente del Mazo (MaterialInspectionCockpit.tsx evolucionado):**
   - Analiza el texto e imágenes de tu tarjeta monolítica original.
   - **Novedad crítica vs. el sistema actual:** El botón *"Analizar y planificar"* ya no analiza en el vacío; cruza los ejes temáticos detectados en la tarjeta contra el almacén del mazo (deck_knowledge_store.py):
     - **Prerrequisitos:** Verifica si los prerrequisitos del tema ya existen como tarjetas verificadas en el mazo. Si falta un prerrequisito básico para un principiante, lo añade automáticamente como un eje a generar en este mismo plan.
     - **Detección de Colisiones con Tarjetas Existentes (Tu Regla #2 - Opción B):** Si la tarjeta monolítica toca los temas *X* e *Y*, y el sistema detecta que en el mazo ya existe otra tarjeta *B* que habla de *Y*:
       - Si *B* es de **buena calidad** (SANITIZED_VERIFIED), el planificador te avisa: *"El eje Y ya está cubierto por la tarjeta verificada #B (Justificación: ...). Omitiremos o ajustaremos este eje para no duplicar"*.
       - Si *B* es de **mala calidad** (FLAGGED_LOW_QUALITY), el sistema **no asume que *Y* ya está aprendido ni mezcla *B* a la fuerza**: te notifica que *B* es defectuosa, genera las tarjetas necesarias para cubrir la tarjeta actual *A* con su justificación pedagógica, y marca a *B* en la cola con un **Vínculo de Memoria Contextual** (covered_by_notes: [nuevas_cards_de_A]).
2. **Generación, Justificación y Auditoría Cognitiva:**
   - Se generan las *N* tarjetas atómicas siguiendo el plan aprobado.
   - Cada tarjeta muestra explícitamente:
     - **Eje temático y nodo del DAG al que pertenece.**
     - **Justificación pedagógica** (por qué existe esta tarjeta y qué micro-habilidad o retención garantiza).
     - **Advertencia de similitud** (si se relaciona con otra tarjeta del lote o del mazo).
3. **Commit Quirúrgico en Anki (Tu Regla #3):**
   - Al aprobar el lote de la tarjeta actual:
     1. Se insertan las *N* tarjetas nuevas verificadas en Anki y se indexan como SANITIZED_VERIFIED en deck_knowledge_store.py.
     2. Se **elimina definitivamente** la tarjeta monolítica original en Anki (deleteNotes).
     3. La barra de progreso de saneamiento avanza al siguiente turno topológico.
4. **Cuando llega el turno de la Tarjeta *B* (la que fue encolada por mala calidad):**
   - Al cargarse *B* en el Cockpit de *"Analizar y planificar"*, el sistema lee su **Vínculo de Memoria Contextual**: sabe exactamente qué tarjetas ya creaste en el turno de *A*.
   - El planificador te dice: *"El 70% de esta tarjeta defectuosa ya fue cubierto con calidad por las tarjetas creadas en el turno anterior; solo falta extraer el concepto Z (1 tarjeta nueva) y eliminar esta nota original"*.

Fase 3: Certificación de Suficiencia, Socratic Red Teaming y Expansión

Cuando un nodo temático del DAG (ej. *Modelo OSI*) tiene el 100% de sus tarjetas asociadas en estado SANITIZED_VERIFIED:

1. **Auditoría de Suficiencia para Principiante (ThematicCoverageChecklist.tsx + deck_intelligence_service.py):**
    - El sistema evalúa dos porcentajes separados:
        - **% de Saneamiento del Nodo:** (Tarjetas limpias vs. tarjetas sucias).
        - **% de Cobertura Pedagógica (Brechas):** ¿Las tarjetas actuales son suficientes para que un principiante entienda el Modelo OSI de principio a fin? Si faltan las capas 5 y 6 o la diferencia entre PDU y SDU, el sistema declara la **Brecha Detectada** y te propone generar las tarjetas faltantes antes de dar el tema por cerrado.
2. **Prueba de Fuego Interactiva (socratic_red_teaming_service.py - Tu Regla #4):**
    - Una vez cubiertas las brechas teóricas en tarjetas, se habilita el reto socrático interactivo: el sistema te plantea un escenario o pregunta de comprensión profunda (ej. diagnóstico o flujo de extremo a extremo) para confirmar que la descomposición de tarjetas realmente construyó un modelo mental sólido.
3. **Desbloqueo de Expansión y Siguiente Etapa (deck_expansion_service.py):**
    - Certificado el nodo sin brechas, el DAG lo marca como **Dominado** y habilita de forma segura la expansión hacia el siguiente nodo o hacia la **Etapa 2 del Plan de Estudios** (AWS, Cisco, OCI).

3. Tres Preguntas Puntuales sobre la Interfaz y Transición (Mientras subes el .md suelto)

Para preparar el plan exacto de refactorización de archivos en cuanto subas KEY_FILES_CONCATENATED.md sin comprimir:

1. **Qué eliminar de la interfaz actual (Modales obsoletos):** Al llevar este flujo unificado a la ventana principal (App.tsx + CurationQueueBanner.tsx + MaterialInspectionCockpit.tsx), ¿confirmas que debemos **deprecar y eliminar** ClusterSanitationModal.tsx y RefactorNoteModal.tsx, dejando AnkiCardPickerModal.tsx únicamente como un selector por si algún día quieres elegir a mano qué tarjeta auditar (cargándola igualmente en este mismo flujo principal)?
2. **Cuándo ejecutar el Reto Socrático (socratic_red_teaming_service.py):** ¿Prefieres que el reto socrático interactivo se active **al completar cada nodo temático del DAG** (ej. al terminar todas las cards de "Modelo OSI") o **al finalizar el saneamiento de toda una Etapa del Plan de Estudios**?
3. **Imágenes dentro de las tarjetas monolíticas:** Mencionaste en el Objetivo 1 que muchas de tus tarjetas monolíticas hechas a mano tienen **imágenes** además de mucho texto. Cuando una tarjeta monolítica con una imagen (ej. un diagrama de arquitectura o tabla en imagen) se divide en 4 tarjetas atómicas: ¿quieres que el sistema conserve la etiqueta <img> original de Anki en las tarjetas hijas donde ese diagrama siga siendo relevante (como apoyo visual en el reverso o contexto), o cómo manejas hoy las imágenes de Anki al refactorizar?

Sube KEY_FILES_CONCATENATED.md directamente como archivo suelto (sin .zip) junto con estas 3 confirmaciones y te entregaré el diagnóstico exacto del código actual y el diseño técnico de cada módulo a intervenir.

User prompt: El archivo que me pediste aqui esta y estas son las respuestas a tus preguntas: 1. confirmo su eliminacion. ademas el AnkiCardPickerModal.tsx tambien confirmo su tratamiento que le sugeriste. 2. se activara al completar cada nodo temático del DAG. 3. se supone que el sistema actual "hereda" dichas

imagenes (pueden ser hasta 8 en una unica card), pero si me gustaria una manera "inteligente de tratar dicha herencia, ya que muchas imagenes hacen parte de un mismo tema.

Response: 1. Radiografía y Auditoría del Código Actual (KEY_FILES_CONCATENATED.md)

Al inspeccionar el código fuente de sanitation_service.py, deck_intelligence_service.py, deck_dag_service.py, deck_expansion_service.py, deck_interview_service.py, routes.py y models.py, queda demostrado con exactitud por qué el flujo actual de **"Sanear Clúster"** no cumple tus objetivos pedagógicos ni de consistencia.

A continuación presento los **6 hallazgos críticos** encontrados directamente en el código actual:

Hallazgo 1: La Cola de Saneamiento (get_sanitation_queue) ignora el orden lógico del DAG

- **Qué hace el código hoy:** En SanitationService.get_sanitation_queue (backend/app/services/sanitation_service.py), las notas pendientes se ordenan únicamente con una función local _sort_priority(n) que agrupa primero quality_status == "bad" (0), luego "preevaluated" (1) y finalmente "unreviewed" (2).
- **Por qué falla tu objetivo:** No invoca DeckDAGService.get_topological_card_order(deck_name). En consecuencia, el sistema puede ponerte a sanear primero una tarjeta avanzada (nivel de profundidad 4 en el DAG) antes de haber saneado las tarjetas de conceptos fundamentales (nivel 0).

Hallazgo 2: "Sanear Clúster" se salta por completo el motor de *"Analizar y Planificar"* (StudyPlanInspection)

- **Qué hace el código hoy:** En ClusterSanitationModal.tsx, el botón *"Generar Propuesta Atómica"* llama a POST /anki/decks/{deck_name}/clusters/{cluster_index}/notes/{note_id}/generate-candidates (generate_note_sanitation_candidates en sanitation_service.py). Este método invoca directamente generator_service.execute_from_text o execute_from_media con un prompt genérico, **sin pasar jamás por /cards/inspect (InspectMaterialService)**.
- **Por qué falla tu objetivo:** Al saltarse StudyPlanInspection, el saneamiento de clúster pierde la capacidad de:
  1. Diagnosticar el self_sufficiency_score de la tarjeta monolítica.
  2. Detectar prerrequisitos faltantes (PrerequisiteItem) para un principiante.
  3. Desglosar la tarjeta monolítica en ejes temáticos explícitos (CoreTheme) con conteo planificado de tarjetas (recommended_cards).
  4. Cruzar esos ejes y prerrequisitos contra el mazo mediante match_study_plan.

Hallazgo 3: Bug destructivo en replace_note_transactionally que deja "huérfanos" los nodos del DAG

- **Qué hace el código hoy:** En SanitationService.replace_note_transactionally (sanitation_service.py), cuando sustituyes una nota monolítica por *N* tarjetas atómicas nuevas, el paso 3 ejecuta: DELETE FROM card_concept_mappings WHERE deck_name = ? AND note_id = ?. Y en el paso 4 guarda los embeddings de las tarjetas nuevas en indexed_notes con quality_status = "optimal", pero **nunca inserta los nuevos created_note_ids en la tabla card_concept_mappings ni actualiza la ontología del DAG**.
- **Por qué falla tu objetivo:** En DeckDAGService.get_deck_dag (deck_dag_service.py), el estado de cada nodo (ConceptNode.status) se calcula mirando las tarjetas vinculadas en card_concept_mappings con role == "target". Como borraste el vínculo de la nota vieja y no

vinculaste las nuevas, el nodo del concepto pasa a tener card_count = 0 y se degrada a estado "UNMAPPED" (is_orphan = True) en lugar de pasar a "SANITIZED".

Hallazgo 4: Asunción negativa sobre tarjetas vecinas de mala calidad en negative_boundaries

- **Qué hace el código hoy:** En get_cluster_sanitation_preview y generate_note_sanitation_candidates (sanitation_service.py), el sistema extrae un descriptor superficial de cada nota hermana del clúster (_extract_note_subtopic_descriptor, que toma los primeros 60 caracteres del anverso) y los inyecta todos en el prompt como: "PROHIBIDO evaluar conceptos vecinos: ...".
- **Por qué falla tu objetivo:** El código actual prohíbe tocar un subtema vecino **incluso cuando la nota hermana que supuestamente lo cubre tiene quality_status == "bad"**. Además, si una nota monolítica *A* toca un tema que colisiona con una nota *B* que está **fuera** de ese clúster de K-Means, el generador ni se entera. Y cuando más adelante llega el turno de sanear la nota defectuosa *B*, el sistema no guarda registro de qué conceptos ya quedaron cubiertos por las tarjetas hijas creadas en el turno de la nota *A*.

Hallazgo 5: Limitación en la Herencia Multimodal de Imágenes (1 a 8 imágenes por nota)

- **Qué hace el código hoy:** En domain/models.py, el modelo Flashcard solo tiene source_image_index: Optional[int] (singular) y source_image: Optional[str] (singular), mientras que CoreTheme sí contempla una lista associated_diagrams: List[str]. Además, en sanitation_service.py, cuando una nota tiene múltiples imágenes, execute_from_media solo pasa media_parts[0][0] como imagen principal y el resto en media_parts sin agrupar qué conjunto de imágenes corresponde a cada eje temático.
- **Por qué falla tu objetivo:** Si una tarjeta monolítica tiene hasta 8 imágenes y 3 de ellas pertenecen a un mismo subtema (por ejemplo, una secuencia de pasos o vistas complementarias de una arquitectura), una tarjeta atómica hija solo puede referenciar un source_image singular o termina heredando imágenes que pertenecen a otro eje temático.

Hallazgo 6: El Contrato de Plan de Estudios (LearningPlanContract) es estático y no valida "Cero Brechas + Reto Socrático" por nodo

- **Qué hace el código hoy:** LearningPlanContract (domain/models.py y deck_interview_service.py) solo almacena 4 campos planos (bloom_level, exclusion_boundary, gap_strategy, operational_depth). No modela etapas evolutivas (ej. Etapa 1: *Fundamentos de Redes Nivel Principiante* → Etapa 2: *Certificación Cloud AWS/Cisco/OCI*). Asimismo, cuando un nodo del DAG pasa a SANITIZED, el sistema solo comprueba que las tarjetas existentes asociadas a ese nodo tengan quality_status == "optimal" (all(st == "optimal" for st in mapped_statuses)), pero **no audita si esas tarjetas son suficientes para que un principiante comprenda el nodo sin brechas**, ni dispara el reto interactivo de socratic_red_teaming_service.py.

2. Arquitectura Oficial Definitiva (Alineada con tus Respuestas)

Con base en tus confirmaciones, así quedará estructurado el **Flujo Oficial Unificado de Saneamiento en la Ventana Principal**:

A. Limpieza de Modales y Unificación en App.tsx

1. **Eliminación definitiva:** Se eliminan ClusterSanitationModal.tsx y RefactorNoteModal.tsx (y sus endpoints huérfanos de refactor in-situ si ya no se usan, dado que toda tarjeta defectuosa se

reemplaza eliminando la nota original en Anki).

2. **Rol de AnkiCardPickerModal.tsx:** Se mantiene únicamente como buscador/selector manual para que, si eliges una nota específica del mazo, esta se cargue en el **Cockpit Principal de Saneamiento (MaterialInspectionCockpit.tsx en App.tsx)** siguiendo exactamente las mismas reglas del mazo.
3. **Cola Topológica Oficial en la Ventana Principal (CurationQueueBanner.tsx + App.tsx):**
    - Al activar el saneamiento de un mazo, SanitationService.get_sanitation_queue ordenará las notas pendientes siguiendo estrictamente:
        1. **Nivel de profundidad topológica en el DAG (depth_level = 0, 1, 2...)** usando DeckDAGService (primero conceptos raíz/fundamentales, luego intermedios, luego avanzados).
        2. Dentro del mismo nivel de profundidad, primero notas con quality_status == "bad" y luego "unreviewed".

B. El Turno de Saneamiento con *"Analizar y Planificar"* Contextualizado

Cada nota de la cola se carga en el cockpit principal (App.tsx) y ejecuta automáticamente (o mediante el botón *"Analizar y planificar"*) el flujo enriquecido de InspectMaterialService + DeckIntelligenceService.match_study_plan:

1. **Desglose en Ejes Temáticos (CoreTheme) y Prerrequisitos (PrerequisiteItem):**
    - El inspector analiza la nota monolítica (texto + hasta 8 imágenes) respetando el nivel del usuario definido en la Etapa Activa del Plan de Estudios (ej. *Principiante*).
2. **Herencia Multimodal Inteligente por Grupos Temáticos de Imágenes (1 a 8 imágenes):**
    - Durante *"Analizar y planificar"*, el modelo multimodal clasifica las imágenes de la nota original (Imagen 1 a Imagen 8) y asigna a cada CoreTheme su subconjunto exacto en associated_diagrams: List[str] (además de una breve descripción de qué aporta cada imagen al eje).
    - Extendemos Flashcard en domain/models.py con source_images: List[str] = Field(default_factory=list) (manteniendo retrocompatibilidad con source_image) para que, si un eje temático o tarjeta atómica requiere 2 o 3 imágenes que forman parte de una misma secuencia o explicación visual, la tarjeta hija herede **solo las imágenes pertinentes a su eje** (y ninguna imagen de otros ejes de la nota monolítica).
3. **Política de Colisiones y Cola con Memoria Contextual (Tu Opción B):**
    - Para cada CoreTheme y PrerequisiteItem detectado en la nota actual *A*, match_study_plan busca en todo el mazo (semantic_search en DeckKnowledgeStore):
        - **Caso 1 — Coincide con una nota existente optimal:** Te muestra la tarjeta existente con su justificación pedagógica y marca el eje como DOMINATED (keep_existing) para no duplicarlo, a menos que represente una faceta complementaria (FACETED_COMPLEMENT según PedagogicalDiscriminatorService).
        - **Caso 2 — Coincide con una nota existente bad o unreviewed (Nota *B*):** El sistema **no bloquea ni prohíbe el tema en el turno actual**. Te notifica que la Nota *B* existe pero tiene deuda cognitiva (COGNITIVE_DEBT), permite que el turno actual de la Nota *A* genere todas las tarjetas atómicas de calidad necesarias (cada una con su campo justification obligatorio), y registra en SQLite una relación de **Cobertura Previa (sanitation_coverage_ledger)** indicando qué conceptos/facetas de la Nota *B* ya fueron cubiertos por las nuevas tarjetas creadas en el turno de *A*.
        - Cuando la cola llegue al turno de la Nota *B*, *"Analizar y planificar"* leerá ese registro, te mostrará las tarjetas recién creadas en el turno de *A* como ya óptimas (DOMINATED), desmarcará los ejes ya cubiertos para no duplicarlos, y solo generará tarjetas para los subtemas exclusivos que aún falten en *B* antes de eliminar la Nota *B* original.
4. **Sustitución Transaccional Completa (replace_note_transactionally reparado):**

- Inserta las nuevas tarjetas atómicas en Anki.
- Elimina definitivamente la nota monolítica original en Anki (delete_notes).
- Indexa los embeddings de las nuevas tarjetas como optimal.
- **Nuevo paso crítico:** Vincula automáticamente las nuevas tarjetas creadas (created_note_ids) a los nodos correspondientes del DAG en card_concept_mappings (role="target") y actualiza los clústeres, para que el progreso del nodo en el DAG suba en tiempo real.

C. Cierre de Nodo del DAG: Suficiencia Pedagógica + Reto Socrático Interactivo

Cuando la última tarjeta asociada a un nodo del DAG (ej. modelo_osi) es saneada (100% tarjetas óptimas en el nodo):

1. **Auditoría Automática de Suficiencia para Principiante (audit_topic_coverage vinculada al Nodo):**
   - El sistema evalúa si las tarjetas óptimas vinculadas a ese nodo cubren el 100% de los micro-conceptos necesarios para un principiante según la Etapa Activa del Plan de Estudios.
   - Si detecta brechas (orphan_gaps), las muestra inmediatamente para generar las tarjetas de cierre del nodo.
2. **Reto Socrático Interactivo (socratic_red_teaming_service.py):**
   - Una vez que el nodo tiene 100% de tarjetas óptimas y 0 brechas, se lanza el modal/panel interactivo de **Certificación Socrática del Nodo**: el agente te hace 1–2 preguntas socráticas de integración/trampa sobre ese nodo temático. Al superarlo, el nodo pasa al nuevo estado **MASTERED (Dominado y Verificado)**, actualizando el porcentaje global de dominio del mazo y desbloqueando los nodos dependientes o la siguiente etapa del Plan de Estudios.

D. Plan de Estudios Dinámico Multi-Etapa (LearningPlanContract Evolucionado)

- Extendemos LearningPlanContract (domain/models.py) para soportar stages: List[CurriculumStage] (Etapa 1: *Fundamento Base de 0 a 100*, Etapa 2: *Especialización / Certificación Cloud AWS, Cisco, OCI*, etc.). Cada etapa define su target_learner_level (beginner, intermediate, advanced), sus temas objetivo y su blueprint opcional (ExamBlueprint).

3. Últimos Archivos Necesarios para Ver el 100% del Código Antes de Modificar

El archivo KEY_FILES_CONCATENATED.md se cortó justo a la mitad de ClusterSanitationModal.tsx (línea ~510 de ese componente), por lo que quedaron fuera del corte los últimos archivos de esa concatenación y 3 servicios del backend que intervienen en el plan.

Por favor, sube en un archivo .md o .txt suelto (sin .zip) los siguientes archivos para completar la lectura del código:

1. **Del Frontend (que quedaron cortados al final de KEY_FILES_CONCATENATED.md):**
   - frontend/src/App.tsx
   - frontend/src/components/MaterialInspectionCockpit.tsx
   - frontend/src/components/DeckStudioModal.tsx
   - frontend/src/components/CurationQueueBanner.tsx y frontend/src/components/ActiveStudyPlanBanner.tsx
2. **Del Backend (para conectar "Analizar y planificar", herencia de imágenes, SQLite y el reto socrático):**
   - backend/app/services/inspect_material_service.py

- backend/app/services/generate_cards_service.py
- backend/app/services/socratic_red_teaming_service.py
- backend/app/adapters/db/deck_knowledge_store.py

En cuanto los subas en texto plano, verificaremos las firmas exactas de esos métodos y pasaremos directamente a la implementación limpia paso a paso.

User prompt: con esto creo que tendras el 100% del contexto, si es asi quiero que diseñes un .md lo mas completo posible sobre la arquitectura oficial que definimos segun mis respuestas, ademas de documentar los hallazgos que deben corregirse.

Response: Con los archivos proporcionados en KEY_FILES_CONCATENATED.md y KEY_FILES_COMPLEMENTARY.md ya contamos con el 100% del contexto del sistema tanto en Backend como en Frontend.

A continuación te entrego el documento arquitectónico oficial completo en formato Markdown (diseñado para reemplazar el antiguo docs/SANITATION_WORKFLOW_AUDIT.md), donde quedan formalizados todos los **hallazgos de código a corregir** y la **Arquitectura Oficial Unificada de Saneamiento, Suficiencia Pedagógica y Expansión Curricular**.

Arquitectura Oficial de Saneamiento Guiado, Suficiencia Pedagógica y Evolución Curricular (v2.0)

> **Estado**: Especificación Oficial Aprobada para Implementación
> **Reemplaza a**: `docs/SANITATION_WORKFLOW_AUDIT.md` (v1)[cite: 3]
> **Alcance**: Unificación del Flujo de Saneamiento en la Ventana Principal (`App.tsx`), Gobernanza por Grafo Acíclico Dirigido (`DAG`), Herencia Multimodal Inteligente (1–8 imágenes), Memoria Contextual de Colisiones, Certificación Socrática por Nodo y Plan de Estudios Dinámico Multi-Etapa.

---

1. Auditoría de Código Actual: Hallazgos Críticos que Deben Corregirse

Tras auditar los servicios del backend y componentes del frontend actuales, se identificaron **9 defectos estructurales y funcionales** que impiden cumplir los objetivos pedagógicos y de experiencia de usuario:

H-01. Desconexión entre el Orden Topológico del DAG y la Cola de Saneamiento (`sanitation_service.py`)
* **Evidencia en código**: En `SanitationService.get_sanitation_queue`, las notas pendientes del mazo se ordenan exclusivamente mediante una función local `_sort_priority(n)` que prioriza el estado `'bad'` (`0`), `'preevaluated'` (`1`) y `'unreviewed'` (`2`)[cite: 3].
* **Impacto**: No se consulta `DeckDAGService.get_topological_card_order(deck_name)`[cite: 3]. Como resultado, el usuario termina saneando tarjetas de conceptos avanzados (hojas del grafo) antes de haber saneado y consolidado las tarjetas de conceptos fundamentales (raíces del grafo con `depth_level = 0`)[cite: 3].

H-02. "Sanear Clúster" omite por completo el motor de *"Analizar y Planificar"* (`sanitation_service.py` y `ClusterSanitationModal.tsx`)
* **Evidencia en código**: En `ClusterSanitationModal.tsx`, el botón *"Generar Propuesta Atómica"* invoca `api.generateClusterNoteCandidates`, el cual llama a

`SanitationService.generate_note_sanitation_candidates`[cite: 3, 4]. Este método ejecuta directamente `generator_service.execute_from_text` o `execute_from_media` sin invocar jamás a `InspectMaterialUseCase` (`/cards/inspect`) ni a `DeckIntelligenceService.match_study_plan` (`/anki/decks/match-study-plan`)[cite: 3, 4].
* **Impacto**: Al usar el flujo de clústeres, el sistema pierde la capacidad de diagnosticar autosuficiencia (`self_sufficiency_score`), detectar prerrequisitos faltantes (`PrerequisiteItem`), dividir la nota monolítica en ejes temáticos explícitos (`CoreTheme`), calcular el número óptimo de tarjetas por eje (`recommended_cards`) y verificar colisiones semánticas contra todo el mazo[cite: 3, 4].

H-03. Pérdida de vínculos con el Grafo (DAG) tras la sustitución transaccional (`sanitation_service.py`)
* **Evidencia en código**: En `SanitationService.replace_note_transactionally` (y en `sanitize_cluster`), tras crear las nuevas notas en Anki y eliminar la nota monolítica original, el Paso 3 ejecuta `DELETE FROM card_concept_mappings WHERE deck_name = ? AND note_id = ?`[cite: 3]. Sin embargo, en el Paso 4 solo inserta las nuevas tarjetas en `indexed_notes` mediante `save_note_embeddings` y **nunca vincula los nuevos `created_note_ids` en la tabla `card_concept_mappings`**[cite: 3].
* **Impacto**: En `DeckDAGService.get_deck_dag`, el estado de cada nodo (`ConceptNode.status`) depende de las tarjetas vinculadas en `card_concept_mappings` con `role == 'target'`[cite: 3]. Al borrar el vínculo de la nota monolítica vieja y no registrar los vínculos de las tarjetas atómicas hijas, el nodo pierde sus tarjetas mapeadas (`card_count = 0`), cae erróneamente al estado `''UNMAPPED'' (`is_orphan = True`) en vez de avanzar a `''SANITIZED''`[cite: 3].

H-04. Asunción negativa sobre notas vecinas de mala calidad en `negative_boundaries` (`sanitation_service.py` y `generate_cards_service.py`)
* **Evidencia en código**: Tanto `get_cluster_sanitation_preview` como `generate_note_sanitation_candidates` extraen descriptores superficiales de las notas hermanas del clúster (`_extract_note_subtopic_descriptor`) e inyectan una directiva rígida en el prompt: `"PROHIBIDO evaluar conceptos vecinos: ..."`[cite: 3]. De igual forma, `GenerateCardsUseCase._apply_mutual_negative_boundaries` inyecta `get_cluster_sibling_boundaries` sin filtrar por la calidad de las notas vecinas[cite: 4].
* **Impacto**: Si una tarjeta monolítica A toca los temas X e Y, y existe otra tarjeta B en el mazo que menciona Y pero es de **mala calidad** (`quality_status == 'bad'` o `'unreviewed'`), el sistema prohíbe generar tarjetas sobre Y asumiendo falsamente que ya está bien cubierto[cite: 3]. Peor aún: si se permite generar Y en el turno de A, cuando llega el turno de procesar la tarjeta defectuosa B, el sistema no recuerda de forma determinista qué subtemas de B ya fueron saneados durante el turno de A[cite: 3, 4].

H-05. Herencia ingenua y desordenada de imágenes en el Frontend y Backend (`App.tsx`, `models.py`, `sanitation_service.py`)
* **Evidencia en código**:
  1. En `domain/models.py`, la entidad `Flashcard` solo posee `source_image_index: Optional[int]` y `source_image: Optional[str]` (singular), impidiendo asociar un grupo de imágenes complementarias a una misma tarjeta[cite: 3].
  2. En `App.tsx` (`handleConfirmGenerateFromInspection`), las imágenes heredadas (`approvedPlan.inherited_media_files`) se reparten con un `mediaToInherit.shift()!` secuencial (una imagen por tarjeta sin importar de qué subtema habla la tarjeta) y las imágenes sobrantes se pegan todas juntas en `generatedCards[0]`[cite: 4].
* **Impacto**: Cuando una nota monolítica tiene hasta 8 imágenes donde varias pertenecen a un mismo eje temático (ej. un flujo de 3 diagramas), el reparto ciego por `.shift()` adjunta diagramas en tarjetas que hablan de otro subtema o amontona el sobrante en la primera tarjeta[cite: 4].

H-06. Eliminación prematura o peligrosa de notas colisionadas en `MaterialInspectionCockpit.tsx`
* **Evidencia en código**: En `MaterialInspectionCockpit.tsx` (`handleApproveInspection`), el código recorre `inspection.prerequisites` e `inspection.core_themes` y, para cualquier coincidencia con `status ===

'cognitive_debt'`, agrega automáticamente su `matched_note_id` a `obsoleteIds` para borrarla de Anki junto con la nota origen[cite: 4].
* **Impacto**: Viola directamente la regla de darle a cada tarjeta existente de mala calidad **su propio turno de procesamiento**. Si la nota colisionada B tenía otros subtemas adicionales que no estaban en la nota A, borrar B durante el turno de A destruye conocimiento del mazo sin haberlo auditado en su propio turno[cite: 4].

H-07. Flujo de Saneamiento fragmentado en modales redundantes y estado huérfano (`App.tsx`, `DeckStudioModal.tsx`, `ClusterSanitationModal.tsx`, `RefactorNoteModal.tsx`)
* **Evidencia en código**:
  1. En `App.tsx`, el modo `isSanitationMode` y `sanitationQueue` están cableados a `sessionStorage.getItem('ankicraft_sanitation_mode')` con un nombre de mazo fijo `'Oracle Cloud'` en el `useEffect` de arranque, sin ningún botón en la interfaz que lo active[cite: 3, 4].
  2. Coexisten dos colas distintas en `App.tsx`: `sanitationQueue` (Fase 3) y `curationQueue` (`CurationQueueBanner.tsx`)[cite: 4].
  3. El saneamiento de clúster vive encerrado en un modal secundario (`ClusterSanitationModal.tsx`) dentro de otro modal (`DeckStudioModal.tsx`), separado de la ventana principal[cite: 4].
  4. `RefactorNoteModal.tsx` mantiene un flujo paralelo de edición *in-situ* (`preview_refactor_note` / `apply_refactor_note`) que ya no aplica porque la política oficial es sustituir y eliminar totalmente la tarjeta defectuosa original[cite: 3, 4].

H-08. Ausencia de validación de "Suficiencia para Principiante" y "Reto Socrático" al cerrar un nodo del DAG (`deck_dag_service.py` y `socratic_red_teaming_service.py`)
* **Evidencia en código**:
  1. En `DeckDAGService.get_deck_dag`, un nodo pasa a `status = "SANITIZED"` simplemente cuando `all(st == "optimal" for st in mapped_statuses)`[cite: 3]. Esto verifica que las tarjetas actuales están bien redactadas, pero **no verifica si son suficientes para que un principiante comprenda el tema completo sin brechas**[cite: 3].
  2. `SocraticRedTeamingService` (`socratic_red_teaming_service.py`) solo se utiliza para auditar la robustez del prompt defensor en tests (`Level 5 Audit`), pero no está conectado a un flujo interactivo con el usuario al completar un nodo del DAG[cite: 3, 4].

H-09. Contrato de Plan de Estudios (`LearningPlanContract`) estático y monolítico (`models.py` y `deck_interview_service.py`)
* **Evidencia en código**: `LearningPlanContract` solo guarda 4 atributos globales (`bloom_level`, `exclusion_boundary`, `gap_strategy`, `operational_depth`) en una fila única por mazo en `deck_learning_contracts`[cite: 3, 4].
* **Impacto**: No permite modelar un objetivo global dinámico con **Etapas Secundarias / Evolutivas** (ej. Etapa 1: *Fundamentos de Redes / Modelo OSI de 0 a 100 para principiante* → Etapa 2: *Certificación Cloud AWS / Cisco / OCI*) ni medir el porcentaje de avance y cobertura por etapa[cite: 3].

---

2. Decisiones Oficiales de Diseño (Invariantes del Sistema)

1. **Unificación Total en la Ventana Principal (`App.tsx`)**:
   - Todo el flujo de saneamiento vive en la ventana principal utilizando una única **Cola Topológica de Saneamiento** y el motor completo de **"Analizar y Planificar" (`MaterialInspectionCockpit.tsx`)**[cite: 4].
   - **Eliminación definitiva de componentes obsoletos**: Se eliminan `ClusterSanitationModal.tsx` y `RefactorNoteModal.tsx` (junto con sus rutas de refactorización *in-place* en backend y referencias en `DeckStudioModal.tsx`)[cite: 3, 4].
   - **Rol de `AnkiCardPickerModal.tsx`**: Se conserva exclusivamente como selector manual para que el

usuario pueda elegir cualquier tarjeta puntual de Anki y cargarla dentro del mismo pipeline contextualizado de *"Analizar y Planificar"* en la ventana principal[cite: 3, 4].
2. **Política de Colisiones con Tarjetas de Mala Calidad (Opción B — Turno Propio con Memoria Contextual)**:
   - Cuando al analizar la Nota A se detecta que un subtema o prerrequisito colisiona con una Nota existente B de mala calidad (`bad` o `unreviewed`), el sistema **nunca asume que el tema ya está aprendido, ni prohíbe generarlo en A, ni elimina B prematuramente**[cite: 3, 4].
   - El sistema notifica la colisión defectuosa, permite generar en el turno de A todas las tarjetas atómicas necesarias (cada una con su justificación pedagógica), encola o mantiene a B en su turno correspondiente de la cola, y registra en SQLite un **Asiento de Cobertura (`sanitation_coverage_ledger`)** que vincula a B con las nuevas tarjetas creadas en el turno de A.
   - Cuando llega el turno propio de la Nota B, el motor de *"Analizar y Planificar"* carga ese historial, marca como `DOMINATED` lo que ya fue cubierto por las tarjetas hijas de A, y solo planifica tarjetas para el contenido exclusivo restante de B.
3. **Política de Destrucción de la Nota Monolítica Original**:
   - Al aprobar las tarjetas atómicas generadas para la nota en turno, la nota monolítica original se **elimina totalmente de Anki Desktop** (`delete_notes`) mediante reemplazo transaccional seguro (solo después de verificar que AnkiConnect insertó con éxito las tarjetas nuevas)[cite: 3].
4. **Cierre de Nodo del DAG: Suficiencia Pedagógica + Reto Socrático Interactivo**:
   - Al completar el saneamiento de todas las notas asociadas a un nodo temático del DAG, el sistema ejecuta automáticamente una **Auditoría de Suficiencia del Nodo** (evaluando si un principiante puede comprender el 100% del nodo con esas tarjetas o si existen brechas)[cite: 3].
   - Una vez cubiertas las brechas (0 brechas pendientes), se activa el **Reto Socrático Interactivo del Nodo** (`socratic_red_teaming_service.py` adaptado a evaluación del estudiante/mazo) antes de certificar el nodo como `MASTERED`[cite: 4].
5. **Plan de Estudios Dinámico Multi-Etapa (Fase 2 Evolucionada)**:
   - El `LearningPlanContract` modela una **Ruta Evolutiva por Etapas (`CurriculumStage`)**, permitiendo tener una Etapa Base activa (ej. *Fundamentos de Redes - Principiante*) y Etapas Secundarias en cola o paralelas (ej. *AWS / Cisco / OCI*), acoplando el nivel del usuario, el DAG y las auditorías de suficiencia a la etapa activa[cite: 3].

---

3. Arquitectura del Flujo Oficial de Saneamiento (Extremo a Extremo)

```mermaid
flowchart TD
    subgraph Fase2 ["1. Gobernanza y Plan de Estudios Dinámico (Fase 2)"]
        LPC[LearningPlanContract Multi-Etapa<br/>Etapa 1: Fundamentos Principiante -> Etapa 2: Cloud/Certificación]
        ONT[Extracción Ontológica + DAG Topológico<br/>DeckDAGService: Raíces depth=0 -> Hojas depth=N]
        LPC --> ONT
    end

    subgraph Cola ["2. Cola Topológica Unificada en Ventana Principal (App.tsx)"]
        ONT --> QUEUE[SanitationService.get_sanitation_queue<br/>Orden: 1° depth_level ASC, 2° bad > unreviewed]
        PICKER[AnkiCardPickerModal<br/>Selección Manual Puntual] --> TURN
        QUEUE --> TURN[Turno Activo en Ventana Principal]
    end
```

```
    subgraph Cockpit ["3. Analizar y Planificar Contextualizado (MaterialInspectionCockpit)"]
        TURN --> INSPECT[InspectMaterialUseCase + Agrupación Multimodal<br/>Asigna imágenes 1..8 por CoreTheme]
        INSPECT --> MATCH[match_study_plan + Consulta de Ledger Contextual]
        MATCH -->|Colisión con Nota Óptima| DOM[Marca Eje como DOMINATED<br/>Muestra tarjeta existente + justificación]
        MATCH -->|Colisión con Nota Mala B| LEDGER[Mantiene Eje ACTIVO en Turno A<br/>Notifica deuda en Nota B y registra en Ledger para el turno de B]
    end

    subgraph Commit ["4. Generación, Auditoría y Sustitución Transaccional"]
        DOM --> GEN[Generación Streaming + Justificación Pedagógica<br/>Herencia Inteligente de Imágenes por Eje]
        LEDGER --> GEN
        GEN --> REPLACE[replace_note_transactionally<br/>1. Crea atómicas en Anki<br/>2. Borra monolito original<br/>3. Vincula nuevos note_ids al DAG en card_concept_mappings<br/>4. Escribe en sanitation_coverage_ledger]
    end

    subgraph CierreNodo ["5. Certificación de Nodo del DAG y Expansión"]
        REPLACE --> CHECK_NODE{¿Nodo del DAG 100%<br/>tarjetas óptimas?}
        CHECK_NODE -->|No| QUEUE
        CHECK_NODE -->|Sí| SUFF[Auditoría de Suficiencia para Principiante<br/>¿Existen brechas para entender el tema de 0 a 100?]
        SUFF -->|Hay Brechas| GAP_GEN[Generar Tarjetas de Brechas del Nodo]
        GAP_GEN --> SUFF
        SUFF -->|0 Brechas| SOCRATIC[Reto Socrático Interactivo del Nodo<br/>SocraticRedTeamingService]
        SOCRATIC -->|Aprobado| MASTERED[Nodo Certificado: MASTERED<br/>Desbloquea dependientes / Etapa 2 del Plan]
    end
```

4. Diseño Detallado de Subsistemas y Modelos de Datos

4.1. Plan de Estudios Dinámico Multi-Etapa (domain/models.py y deck_interview_service.py)

Se extiende LearningPlanContract para soportar etapas evolutivas sin romper compatibilidad hacia atrás:

```
class LearnerExpertiseLevel(str, Enum):
    BEGINNER = "beginner"         # Principiante absoluto: requiere definir cada término desde primeros principios
    INTERMEDIATE = "intermediate" # Conoce fundamentos base; foco en mecanismos y arquitectura
    ADVANCED = "advanced"         # Foco en casos borde, optimización y troubleshooting

class CurriculumStage(BaseModel):
    stage_id: str = Field(..., description="Ej: 'stage_1_foundations', 'stage_2_oci_architect'")
    order_index: int = Field(..., ge=1)
    title: str = Field(..., description="Ej: 'Etapa 1: Fundamentos de Redes y Modelo OSI (0 a 100)'")
    target_learner_level: LearnerExpertiseLevel = LearnerExpertiseLevel.BEGINNER
    bloom_level: str = Field(default="Comprensión y Mecanismos Causales")
```

core_domains: List[str] = Field(default_factory=list, description="Temas núcleo de esta etapa")
exclusion_boundary: List[str] = Field(default_factory=list)
blueprint_exam_name: Optional[str] = None
status: str = Field(default="active", description="'active' | 'locked' | 'completed'")
sanitation_progress_pct: float = 0.0
sufficiency_coverage_pct: float = 0.0

- **Comportamiento en InspectMaterialUseCase y GenerateCardsUseCase**:
    - Cuando la etapa activa tiene target_learner_level == "beginner", el prompt de inspección e instrucción pedagógica obliga al modelo a **no dar por sentado ningún acrónimo, capa o protocolo**, exigiendo que toda tarjeta monolítica se desglose desde sus definiciones base hasta su mecánica operativa.
    - Cuando todos los nodos del DAG pertenecientes a la Etapa 1 alcanzan el estado MASTERED, el sistema notifica que es **100% seguro expandir el mazo** y activa automáticamente la Etapa 2 (ej. AWS, Cisco, OCI).

4.2. Cola Oficial de Saneamiento con Ordenamiento Topológico Estricto (sanitation_service.py)

Se corrige SanitationService.get_sanitation_queue(deck_name):

1. Consulta DeckDAGService.get_deck_dag(deck_name) y obtiene para cada note_id el **mínimo depth_level** de los nodos conceptuales a los que está mapeada (card_concept_mappings), además de su posición en get_topological_card_order(deck_name).
2. Si una nota aún no tiene extracción ontológica en note_extracted_knowledge, dispara una extracción ligera o la ubica tras las raíces conocidas.
3. Ordena pending_notes con la tupla de prioridad:
   def _topological_sanitation_sort_key(note: DeckNoteRecord) -> Tuple[int, int, int, int]:
   is_in_active_stage = 0 if note_in_active_stage(note.note_id) else 1
   node_depth = note_min_depth_map.get(note.note_id, 999)
   quality_prio = 0 if note.quality_status == "bad" else (1 if note.quality_status == "unreviewed" else 2)
   topo_idx = topo_rank_map.get(note.note_id, 999999)
   return (is_in_active_stage, node_depth, quality_prio, topo_idx)

4. Esto garantiza que el usuario **siempre sanee primero los conceptos iniciales (depth_level = 0)** antes de escalar a conceptos dependientes (depth_level = 1, 2, ...).

4.3. Herencia Multimodal Inteligente de Imágenes (1 a 8 Imágenes por Nota)

Para resolver la herencia de hasta 8 imágenes donde varias pertenecen a un mismo tema:

1. **Modelo de Datos (domain/models.py)**:
    - En CoreTheme: se añade diagram_roles: Dict[str, str] = Field(default_factory=dict, description="Mapa filename -> explicación de qué aporta esta imagen al eje") junto a associated_diagrams: List[str].
    - En Flashcard: se añade source_images: List[str] = Field(default_factory=list, description="Lista de imágenes de Anki heredadas por esta tarjeta como soporte visual coherente") (manteniendo source_image como alias del primer elemento para retrocompatibilidad).
2. **Fase de Inspección (InspectMaterialUseCase + gemini_card_generator_adapter.py)**:
    - Al ejecutar *"Analizar y Planificar"* sobre una nota con M imágenes ($1 \leq M \leq 8$), todas las imágenes se envían etiquetadas explícitamente con su nombre de archivo exacto ([IMAGEN_1:

nombre_archivo.png], [IMAGEN_2: ...]).
    - El LLM agrupa las imágenes temáticamente y asigna a cada CoreTheme el subconjunto exacto de archivos en associated_diagrams (ej. si las imágenes 2, 3 y 4 explican el *Handshake TCP*, las 3 quedan vinculadas a ese único CoreTheme).
    - En MaterialInspectionCockpit.tsx, el usuario ve qué imágenes quedaron agrupadas en cada eje temático y puede mover o quitar una imagen de un eje con un clic antes de generar.
3. **Fase de Generación y Renderizado (GenerateCardsUseCase y App.tsx)**:
    - Cada tarjeta generada para un CoreTheme recibe en source_images **únicamente** las imágenes que el modelo determinó pertinentes dentro de theme.associated_diagrams (pudiendo ser 0, 1 o varias imágenes si forman una unidad visual indivisible).
    - Se **elimina de App.tsx el bloque mediaToInherit.shift()!** que repartía imágenes a ciegas. En su lugar, el contenedor visual de la tarjeta inyecta limpiamente en el campo explanation la galería de card.source_images.

4.4. Política de Colisiones y Memoria Contextual Diferida (sanitation_coverage_ledger)

Para cumplir estrictamente tu **Regla #2 (Opción B)** sin asumir negativamente sobre tarjetas existentes de mala calidad:

1. **Nueva Tabla en SQLite (backend/app/adapters/db/deck_knowledge_store.py)**:
```
CREATE TABLE IF NOT EXISTS sanitation_coverage_ledger (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    deck_name TEXT NOT NULL,
    source_sanitized_note_id INTEGER NOT NULL,
    colliding_pending_note_id INTEGER NOT NULL,
    covered_concept_title TEXT NOT NULL,
    covered_facet_summary TEXT,
    created_atomic_note_ids TEXT NOT NULL, -- JSON list of new optimal note IDs
    created_at TEXT NOT NULL
);
CREATE INDEX IF NOT EXISTS idx_ledger_pending_note
    ON sanitation_coverage_ledger(deck_name, colliding_pending_note_id);
```

2. **Qué ocurre en el Turno de la Nota *A* (cuando colisiona con la Nota *B* de mala calidad)**:
    - match_study_plan detecta que el eje *Y* de la Nota *A* tiene alta similitud (≥0.68) con la Nota existente *B*.
    - Si la Nota *B* tiene quality_status == "optimal", el eje *Y* se marca como DOMINATED (desmarcado por defecto para no duplicar, mostrando qué tarjeta óptima ya lo cubre y por qué).
    - Si la Nota *B* tiene quality_status in ("bad", "unreviewed"):
        - El eje *Y* **permanece seleccionado (selected = True)** en el plan de la Nota *A*.
        - **NO se agrega el ID de la Nota *B* a obsolete_note_ids** (corrigiendo el bug de MaterialInspectionCockpit.tsx que borraba *B* sin darle su propio turno).
        - En la interfaz del eje *Y* se muestra el aviso: *"⚠️ Colisión con Nota defectuosa #B: Se generarán las tarjetas de calidad para este eje ahora, y la Nota #B quedará marcada en su turno de la cola con este subtema ya resuelto"*.
    - Al ejecutar replace_note_transactionally para la Nota *A*, el backend registra en sanitation_coverage_ledger que el subtema *Y* de la Nota *B* ya fue cubierto por los created_note_ids de *A*, y asegura que la Nota *B* tenga estado 'bad' en la cola para ser procesada en su turno.
3. **Qué ocurre cuando llega el Turno de la Nota *B***:

- Al abrir *"Analizar y Planificar"* para la Nota *B*, InspectMaterialUseCase + match_study_plan consultan sanitation_coverage_ledger para colliding_pending_note_id = B.note_id.
- Los ejes de *B* que ya fueron cubiertos en el turno de *A* aparecen automáticamente como 🟢 **Cubierto en Turno Previo (Notas #...)** (selected = False, recommended_cards = 0), mientras que cualquier concepto exclusivo de *B* que no estuviera en *A* queda activo (selected = True) para generar sus tarjetas faltantes antes de eliminar *B* de Anki.

4.5. Reparación de replace_note_transactionally y Sincronización Viva del DAG (sanitation_service.py)

Al aprobar las tarjetas atómicas de una nota en el flujo principal:

1. Se crean las nuevas tarjetas en Anki vía anki_bridge.create_notes.
2. Se verifica que created_note_ids no esté vacío (garantía de cero pérdida de datos) y se elimina definitivamente la nota monolítica original con anki_bridge.delete_notes([source_note_id]).
3. **Transferencia y Actualización de Vínculos en el DAG**:
   - Antes de borrar las filas de source_note_id en card_concept_mappings, el servicio recupera todos los concept_id a los que estaba asociada la nota original (más los conceptos de los selected_themes aprobados en el plan).
   - Vincula cada uno de los nuevos created_note_ids a sus respectivos nodos en card_concept_mappings con role = "target".
   - Calcula y guarda los embeddings vectoriales reales de las nuevas tarjetas en indexed_notes con quality_status = "optimal".
   - Re-calcula el estado del nodo en DeckDAGService.get_deck_dag(deck_name, recompute_depths=True).

4.6. Métricas Post-Saneamiento, Auditoría de Suficiencia para Principiante y Reto Socrático por Nodo

Para que el sistema sepa en todo momento el porcentaje de saneamiento, la calidad y el nivel de abordaje de los temas para un usuario principiante:

1. **Tres Métricas Oficiales Separadas por Nodo y por Mazo**:
   - **% de Saneamiento Técnico (sanitation_pct)**: $\frac{\text{Total de Notas del Nodo/Mazo}}{\text{Notas en estado Optimal}} \times 100$.
   - **% de Suficiencia Pedagógica (sufficiency_pct)**: Calculado por DeckIntelligenceService.audit_topic_coverage contrastando las tarjetas óptimas actuales del nodo contra el mapa completo de micro-conceptos requeridos para que un **principiante** domine el tema de 0 a 100.
   - **Estado de Maestría del Nodo (ConceptNode.status)**: Evoluciona en 5 estados formales: UNMAPPED (0 tarjetas) → RAW (tiene tarjetas sin sanear) → SANITIZED (100% tarjetas óptimas, pendiente validar brechas) → SUFFICIENT (100% cobertura sin brechas para principiante) → MASTERED (Reto Socrático superado).
2. **Disparo Automático al Completar un Nodo del DAG**:
   - Cuando la última tarjeta de un nodo temático (ej. *Modelo OSI*) es saneada y el nodo pasa a SANITIZED:
     - **Paso A (Evaluación de Brechas para Principiante)**: El sistema ejecuta automáticamente la auditoría de suficiencia sobre ese nodo considerando que el usuario es principiante. Si detecta brechas (orphan_gaps, por ejemplo: *"Falta explicar cómo interactúa la capa de Red con la de Enlace de Datos en el encapsulamiento"*), las presenta en el mismo cockpit para generar las tarjetas de cierre.

        - **Paso B (Reto Socrático Interactivo — SocraticNodeMasteryModal)**: Cuando el nodo tiene 0 brechas (SUFFICIENT), se despliega el diálogo socrático interactivo impulsado por SocraticRedTeamingService. El agente plantea un escenario integrador o trampa conceptual sobre ese nodo para verificar tanto la solidez del conjunto de tarjetas como la comprensión del tema; al aprobarlo, el nodo se sella como **MASTERED** y habilita el avance al siguiente nodo topológico o a la siguiente etapa del Plan de Estudios.

5. Plan de Intervención por Archivo (Mapa de Implementación)

A. Archivos a Eliminar (Código Muerto y Modales Redundantes)

1. frontend/src/components/ClusterSanitationModal.tsx — **ELIMINAR** (absorbido por el flujo unificado en la ventana principal).
2. frontend/src/components/RefactorNoteModal.tsx — **ELIMINAR** (se reemplaza siempre eliminando la nota original en Anki).

B. Archivos del Backend a Modificar

1. backend/app/domain/models.py:
    - Añadir source_images: List[str] en Flashcard y diagram_roles: Dict[str, str] en CoreTheme.
    - Añadir CurriculumStage, LearnerExpertiseLevel y métricas de etapa en LearningPlanContract.
    - Añadir campos de memoria contextual (ledger_entries) y métricas de progreso por nodo en SanitationQueueResponse y TransactionalReplaceResponse.
2. backend/app/adapters/db/deck_knowledge_store.py:
    - Crear tabla sanitation_coverage_ledger y métodos add_ledger_entry, get_ledger_entries_for_note, clear_ledger_for_note.
3. backend/app/services/sanitation_service.py:
    - Reescribir get_sanitation_queue para ordenar por precedencia topológica del DAG (depth_level + topological_order).
    - Reparar replace_note_transactionally para computar embeddings reales, preservar/actualizar vínculos en card_concept_mappings, registrar asientos en sanitation_coverage_ledger y devolver si el nodo del DAG actual acaba de completarse (completed_dag_node).
    - Deprecar métodos exclusivos del modal de clúster (sanitize_cluster, generate_note_sanitation_candidates).
4. backend/app/services/inspect_material_service.py y backend/app/services/deck_intelligence_service.py:
    - Inyectar nivel de experiencia del usuario (beginner / intermediate / advanced) desde la etapa activa del contrato.
    - Agrupar imágenes (1 a 8) por CoreTheme durante la inspección multimodal.
    - Integrar la lectura de sanitation_coverage_ledger dentro de match_study_plan para marcar como ya cubiertos los ejes resueltos en turnos anteriores.
5. backend/app/services/socratic_red_teaming_service.py y backend/app/api/routes.py:
    - Exponer endpoint interactivo POST /anki/decks/{deck_name}/nodes/{concept_id}/socratic-challenge para ejecutar el reto socrático al completar un nodo del DAG y certificarlo como MASTERED.

C. Archivos del Frontend a Modificar

1. frontend/src/App.tsx y frontend/src/components/CurationQueueBanner.tsx:
    - Unificar sanitationQueue y curationQueue en un único **Banner / Panel de Cola Topológica de**

Saneamiento en la ventana principal, mostrando progreso global del mazo, nodo actual del DAG (Nivel 0: Fundamentos -> ...), botón *"Analizar y Planificar Siguiente Turno"* y barra de sustitución transaccional.
    - Reemplazar el reparto ciego de imágenes (.shift()) por la asignación basada en card.source_images y theme.associated_diagrams.
2. frontend/src/components/MaterialInspectionCockpit.tsx:
    - Eliminar el agregado automático de notas colisionadas a obsoleteIds (garantizando que cada nota existente con deuda tenga su propio turno en la cola).
    - Mostrar las insignias del sanitation_coverage_ledger (*"Cubierto en turno previo"*) y permitir editar qué grupo de imágenes (1..8) hereda cada CoreTheme.
3. frontend/src/components/DeckStudioModal.tsx:
    - Eliminar importaciones y estados de ClusterSanitationModal y RefactorNoteModal.
    - Hacer que cualquier acción de saneamiento en el estudio (*"Iniciar Saneamiento Topológico del Mazo"* o seleccionar una tarjeta con deuda) cierre el modal y active la cola oficial en la ventana principal (App.tsx).

Especificación Oficial: Arquitectura de Saneamiento Guiado, Suficiencia Pedagógica y Evolución Curricular (v2.0)

- **Archivo destino en el proyecto:** docs/SANITATION_WORKFLOW_AUDIT.md
- **Fecha:** 13 de octubre de 2026
- **Estado:** Especificación Oficial Aprobada para Implementación (Reemplaza la arquitectura v1)

1. Diagnóstico del Código Actual: 9 Hallazgos Críticos a Corregir

Tras auditar el código fuente completo del backend y del frontend, se documentan los 9 defectos estructurales que impiden cumplir los objetivos de calidad neurocognitiva y consistencia curricular:

- **H-01 — La Cola de Saneamiento ignora el orden lógico del Grafo (sanitation_service.py):**
    - **Evidencia:** En SanitationService.get_sanitation_queue, las notas pendientes se ordenan únicamente con una función local _sort_priority(n) que agrupa primero quality_status == "bad" (0), luego "preevaluated" (1) y finalmente "unreviewed" (2).
    - **Problema:** No invoca DeckDAGService.get_topological_card_order(deck_name). El usuario termina saneando tarjetas de conceptos avanzados (hojas del grafo) antes de haber saneado los conceptos fundamentales (depth_level = 0).
- **H-02 — "Sanear Clúster" omite el motor de *"Analizar y Planificar"* (sanitation_service.py y ClusterSanitationModal.tsx):**
    - **Evidencia:** En ClusterSanitationModal.tsx, el botón *"Generar Propuesta Atómica"* llama a api.generateClusterNoteCandidates, que ejecuta SanitationService.generate_note_sanitation_candidates. Dicho método invoca directamente generator_service.execute_from_text o execute_from_media sin pasar por InspectMaterialUseCase (/cards/inspect) ni por DeckIntelligenceService.match_study_plan (/anki/decks/match-study-plan).
    - **Problema:** Se pierde la evaluación de autosuficiencia (self_sufficiency_score), la detección de

prerrequisitos (PrerequisiteItem), la partición en ejes temáticos (CoreTheme), el cálculo de densidad de tarjetas por eje (recommended_cards) y el cruce semántico contra todo el mazo.

- **H-03 — Pérdida de vínculos con el Grafo (DAG) tras la sustitución transaccional (sanitation_service.py):**
  - **Evidencia:** En SanitationService.replace_note_transactionally (y en sanitize_cluster), tras crear las nuevas notas en Anki y eliminar la nota monolítica original, el Paso 3 ejecuta DELETE FROM card_concept_mappings WHERE deck_name = ? AND note_id = ?. En el Paso 4 guarda las nuevas tarjetas en indexed_notes, pero **nunca inserta los nuevos created_note_ids en card_concept_mappings**.
  - **Problema:** En DeckDAGService.get_deck_dag, el estado de cada nodo (ConceptNode.status) se calcula evaluando las tarjetas mapeadas en card_concept_mappings con role == "target". Al borrar el vínculo de la nota monolítica original y no vincular las tarjetas atómicas nuevas, el nodo queda con card_count = 0 y cae erróneamente a estado "UNMAPPED" (is_orphan = True) en lugar de avanzar a "SANITIZED".
- **H-04 — Asunción negativa sobre notas vecinas de mala calidad en negative_boundaries (sanitation_service.py y generate_cards_service.py):**
  - **Evidencia:** get_cluster_sanitation_preview y generate_note_sanitation_candidates extraen descriptores superficiales de las notas hermanas del clúster (_extract_note_subtopic_descriptor) e inyectan en el prompt "PROHIBIDO evaluar conceptos vecinos: ..." sin filtrar si esas notas vecinas son de mala calidad. De igual modo, GenerateCardsUseCase._apply_mutual_negative_boundaries inyecta get_cluster_sibling_boundaries sin verificar el quality_status de las notas vecinas.
  - **Problema:** Si una nota monolítica *A* toca los temas *X* e *Y*, y existe otra nota *B* que menciona *Y* pero tiene quality_status == "bad" o "unreviewed", el sistema prohíbe evaluar *Y* asumiendo falsamente que ya está cubierto con calidad. Además, no existe un registro persistente que recuerde en el turno futuro de *B* qué subtemas de *B* ya fueron cubiertos por las tarjetas creadas en el turno de *A*.
- **H-05 — Herencia ciega de imágenes por .shift() secuencial (App.tsx y models.py):**
  - **Evidencia:** En domain/models.py, Flashcard solo define source_image_index: Optional[int] y source_image: Optional[str] (singular). En App.tsx (handleConfirmGenerateFromInspection), las imágenes heredadas (approvedPlan.inherited_media_files) se reparten secuencialmente con mediaToInherit.shift()! a la primera tarjeta que no tenga <img, y las imágenes sobrantes se concatenan todas en generatedCards[0].
  - **Problema:** Cuando una nota monolítica contiene hasta 8 imágenes y varias pertenecen a un mismo subtema, el reparto ciego asigna diagramas a tarjetas de otros subtemas o amontona las imágenes sobrantes en la primera tarjeta.
- **H-06 — Eliminación prematura de notas colisionadas sin respetar su propio turno (MaterialInspectionCockpit.tsx):**
  - **Evidencia:** En MaterialInspectionCockpit.tsx (handleApproveInspection), el código recorre inspection.prerequisites e inspection.core_themes y, para cualquier coincidencia con status === 'cognitive_debt', añade su matched_note_id a obsoleteIds para borrarla de Anki junto con la nota origen actual.
  - **Problema:** Destruye tarjetas existentes que tenían deuda cognitiva sin darles su propio turno de análisis y sin verificar si contenían otros subtemas adicionales que no estaban en la tarjeta actual.
- **H-07 — Fragmentación de la interfaz en modales redundantes y código huérfano (App.tsx, DeckStudioModal.tsx, ClusterSanitationModal.tsx, RefactorNoteModal.tsx):**
  - **Evidencia:** App.tsx mantiene un estado isSanitationMode dependiente de sessionStorage.getItem('ankicraft_sanitation_mode') que pide la cola del mazo fijo 'Oracle Cloud' al iniciar y no tiene botón de activación en la UI. Además, coexisten dos colas separadas (sanitationQueue y curationQueue), el modal ClusterSanitationModal.tsx vive anidado dentro de DeckStudioModal.tsx, y RefactorNoteModal.tsx ofrece una edición *in-situ* que contradice la

regla de eliminación total de la nota monolítica original.

- **H-08 — Falta de auditoría de "Suficiencia para Principiante" y "Reto Socrático" al cerrar un nodo del DAG (deck_dag_service.py y socratic_red_teaming_service.py):**
  - **Evidencia:** En DeckDAGService.get_deck_dag, un nodo recibe status = "SANITIZED" únicamente cuando all(st == "optimal" for st in mapped_statuses). Por su parte, SocraticRedTeamingService solo se usa para pruebas de estrés del prompt defensor (Level 5 Audit) sin exponer un reto interactivo al usuario al completar un nodo del DAG.
  - **Problema:** Tener el 100% de las tarjetas actuales de un nodo en estado "optimal" no garantiza que sean suficientes para que un principiante entienda el tema sin brechas (ej. Modelo OSI completo).
- **H-09 — Contrato de Plan de Estudios (LearningPlanContract) estático y de una sola etapa (models.py y deck_interview_service.py):**
  - **Evidencia:** LearningPlanContract solo guarda bloom_level, exclusion_boundary, gap_strategy y operational_depth como valores planos para todo el mazo.
  - **Problema:** No permite estructurar un objetivo global dinámico con **Etapas Evolutivas** (ej. Etapa 1: *Fundamentos de Redes 0 a 100 para principiante* → Etapa 2: *Ámbitos Cloud como AWS, Cisco, OCI y certificaciones*).

2. Decisiones Oficiales de Arquitectura (Reglas de Oro del Sistema)

1. **Unificación en la Ventana Principal (App.tsx):**
   - Todo el saneamiento del mazo ocurre en la ventana principal combinando la **Cola Topológica Unificada** (con barra de progreso del mazo y del nodo actual) y el cockpit de **"Analizar y Planificar" (MaterialInspectionCockpit.tsx)**.
   - **Eliminación confirmada:** Se eliminan del sistema ClusterSanitationModal.tsx y RefactorNoteModal.tsx.
   - **Rol de AnkiCardPickerModal.tsx:** Se conserva únicamente como selector manual para que, si el usuario elige una nota puntual de Anki, esta se cargue directamente en el mismo flujo contextualizado de *"Analizar y Planificar"* en la ventana principal.
2. **Tratamiento de Colisiones con Tarjetas Existentes de Mala Calidad (Opción B Confirmada):**
   - Cuando la nota en turno *A* toca un tema que colisiona con una nota existente *B* de mala calidad (bad o unreviewed), el sistema **notifica la existencia y mala calidad de *B*, NO asume negativamente que el tema ya está cubierto, NO borra *B* prematuramente, y genera en el turno de *A* todas las tarjetas de calidad necesarias con su justificación pedagógica**.
   - La nota *B* conserva su propio turno en la cola de saneamiento y recibe un registro en el **Ledger de Cobertura (sanitation_coverage_ledger)** para que, cuando llegue su turno, el sistema sepa qué conceptos de *B* ya fueron cubiertos por las tarjetas creadas en el turno de *A*.
3. **Eliminación Total de la Nota Monolítica Original en Anki:**
   - Al aprobar el lote de tarjetas atómicas de la nota en turno, el sistema inserta las nuevas tarjetas en Anki y **elimina definitivamente la nota monolítica original (delete_notes)** sin mantener historial ni archivarla.
4. **Certificación por Nodo del DAG (Cero Brechas + Reto Socrático Interactivo):**
   - Al terminar de sanear las tarjetas asociadas a un nodo temático del DAG, se ejecuta automáticamente una **Auditoría de Suficiencia para Principiante** (detectando si existen brechas para comprender el tema de 0 a 100) seguida de un **Reto Socrático Interactivo** al completar cada nodo temático del DAG.
5. **Herencia Multimodal Inteligente por Agrupación Temática (1 a 8 imágenes):**
   - Durante *"Analizar y Planificar"*, el sistema agrupa las imágenes de la nota original (hasta 8 imágenes) según el eje temático (CoreTheme) al que pertenecen y permite que cada tarjeta atómica hija herede en source_images: List[str] el subconjunto exacto de imágenes relevantes

para su subtema.

6. **Objetivos Dinámicos Multi-Etapa en el Plan de Estudios (Fase 2):**
    - El contrato del mazo (LearningPlanContract) evoluciona a una estructura por etapas (stages: List[CurriculumStage]), donde la Etapa 1 (ej. *Fundamentos de Redes - Nivel Principiante*) gobierna el saneamiento y cierre de brechas inicial, y al completarse habilita de forma segura la expansión hacia las Etapas Secundarias (ej. *AWS, Cisco, OCI, Certificaciones*).

3. Especificación Técnica Detallada por Subsistema

3.1. Modelo de Plan de Estudios Dinámico Multi-Etapa (backend/app/domain/models.py)

Se amplía LearningPlanContract manteniendo compatibilidad con la base de datos SQLite (deck_learning_contracts):

- LearnerExpertiseLevel: Enum con valores "beginner", "intermediate", "advanced".
- CurriculumStage:
    - stage_id: str (ej. "stage_1_foundations", "stage_2_cloud_arch")
    - order_index: int (1, 2, ...)
    - title: str (ej. "Etapa 1: Fundamentos de Redes y Modelo OSI (0 a 100)")
    - target_learner_level: LearnerExpertiseLevel (por defecto "beginner" para asegurar explicaciones desde primeros principios)
    - bloom_level: str
    - core_domains: List[str]
    - exclusion_boundary: List[str]
    - blueprint_exam_name: Optional[str]
    - status: str ("active", "queued", "completed")
    - sanitation_progress_pct: float
    - sufficiency_coverage_pct: float

3.2. Ordenamiento Topológico Estricto de la Cola de Saneamiento (backend/app/services/sanitation_service.py)

En SanitationService.get_sanitation_queue(deck_name):

1. Se consulta DeckDAGService.get_deck_dag(deck_name) para obtener los nodos (ConceptNode) con su depth_level (0 = raíces/fundamentos sin prerrequisitos, 1 = nivel intermedio, etc.) y get_topological_card_order(deck_name).
2. Para cada nota pendiente (quality_status != "optimal"), se calcula su min_depth_level según los nodos a los que está vinculada en card_concept_mappings.
3. La cola se ordena por:
    1. Pertenencia a la Etapa Activa del Plan de Estudios (0 si pertenece a la etapa activa, 1 en caso contrario).
    2. min_depth_level ASC (primero conceptos iniciales depth_level = 0, luego 1, 2, etc.).
    3. Severidad cognitiva (0 para "bad", 1 para "unreviewed", 2 para "preevaluated").
    4. Índice exacto en topological_order.

3.3. Memoria Contextual Diferida para Colisiones (sanitation_coverage_ledger

en deck_knowledge_store.py)

Se añade en DeckKnowledgeStore._init_db() (backend/app/adapters/db/deck_knowledge_store.py) la tabla sanitation_coverage_ledger:

- **Columnas:** id, deck_name, source_sanitized_note_id, colliding_pending_note_id, covered_concept_title, covered_facet_summary, created_atomic_note_ids (JSON), created_at.
- **Flujo en el Turno de la Nota *A*:**
    1. Cuando DeckIntelligenceService.match_study_plan evalúa los ejes temáticos (CoreTheme) de la Nota *A* contra el mazo:
        - Si colisiona con una nota "optimal", marca el eje como DOMINATED (selected = False) y muestra la justificación y el anverso de la tarjeta óptima existente.
        - Si colisiona con una nota *B* en estado "bad" o "unreviewed", clasifica la coincidencia como COGNITIVE_DEBT, **mantiene el eje seleccionado (selected = True) en el plan de *A***, **no incluye a *B* en obsolete_note_ids**, y adjunta una alerta informativa indicando que *B* será procesada en su propio turno.
    2. Cuando el usuario aprueba y sustituye la Nota *A* (replace_note_transactionally), el backend guarda un registro en sanitation_coverage_ledger apuntando a colliding_pending_note_id = B.note_id con los created_note_ids generados en el turno de *A*.
- **Flujo cuando llega el Turno de la Nota *B*:**
    1. Al abrir *"Analizar y Planificar"* sobre la Nota *B*, match_study_plan consulta get_ledger_entries_for_note(deck_name, B.note_id).
    2. Los subtemas de *B* que ya fueron cubiertos por las tarjetas atómicas creadas en el turno de *A* se marcan automáticamente como DOMINATED (mostrando las tarjetas creadas en el turno de *A* y su justificación), dejando seleccionados únicamente los subtemas restantes exclusivos de *B*.

3.4. Herencia Multimodal Inteligente de 1 a 8 Imágenes (inspect_material_service.py, generate_cards_service.py, App.tsx)

1. En domain/models.py:
    - CoreTheme incorpora diagram_roles: Dict[str, str] = Field(default_factory=dict) junto a associated_diagrams: List[str].
    - Flashcard incorpora source_images: List[str] = Field(default_factory=list).
2. En InspectMaterialUseCase.execute (backend/app/services/inspect_material_service.py):
    - Recupera de AnkiConnect hasta las 8 imágenes embebidas en la nota (retrieve_media_base64) e inyecta sus nombres exactos en el prompt multimodal para que Gemini agrupe qué imágenes pertenecen a cada CoreTheme.
3. En MaterialInspectionCockpit.tsx:
    - Cada tarjeta de CoreTheme muestra las miniaturas de sus associated_diagrams y permite activar/desactivar qué imágenes hereda cada eje temático antes de generar.
4. En GenerateCardsUseCase y App.tsx:
    - Se elimina la lógica mediaToInherit.shift()! de App.tsx. Cada tarjeta generada recibe únicamente las imágenes de card.source_images correspondientes a su eje temático.

3.5. Reparación del Vínculo con el DAG en replace_note_transactionally (sanitation_service.py)

En SanitationService.replace_note_transactionally:

1. Antes de ejecutar DELETE FROM card_concept_mappings WHERE deck_name = ? AND note_id = ?, se consultan los concept_id asociados a source_note_id (y los conceptos de los ejes seleccionados).
2. Tras crear las nuevas tarjetas en Anki (create_notes) y borrar la nota monolítica original (delete_notes([source_note_id])), se calculan los embeddings vectoriales de las nuevas tarjetas (_embedding_port.embed_texts), se guardan en indexed_notes con quality_status = "optimal", y se insertan los nuevos vínculos (deck_name, new_note_id, concept_id, 'target') en card_concept_mappings.
3. Se recalcula el DAG (get_deck_dag(deck_name, recompute_depths=True)) y se verifica si alguno de los nodos vinculados acaba de alcanzar el 100% de sus tarjetas en estado "optimal".

3.6. Auditoría de Suficiencia del Nodo y Reto Socrático Interactivo (deck_intelligence_service.py y socratic_red_teaming_service.py)

Cuando un nodo del DAG alcanza el 100% de sus tarjetas mapeadas en estado "optimal":

1. **Auditoría Automática de Suficiencia (audit_node_sufficiency):**
   - Invoca DeckIntelligenceService.audit_topic_coverage sobre el concepto del nodo, especificando el nivel del usuario (beginner en Etapa 1).
   - Evalúa: *"¿Son suficientes estas tarjetas para que un principiante comprenda el tema de principio a fin sin lagunas?"*.
   - Si orphan_gaps no está vacío, el nodo permanece en estado "SANITIZED" (con brechas pendientes) y la interfaz ofrece generar inmediatamente las tarjetas de las brechas detectadas.
2. **Reto Socrático Interactivo del Nodo (SocraticRedTeamingService):**
   - Cuando el nodo tiene 0 brechas pendientes (status = "SUFFICIENT"), se activa en la interfaz el **Reto Socrático del Nodo**.
   - El servicio genera una pregunta socrática de integración o caso de trampa frecuente sobre el nodo; una vez validada la respuesta o defensa conceptual, el nodo pasa a estado **"MASTERED"**, actualizando las barras de progreso del mazo y desbloqueando los nodos dependientes en el DAG.

4. Bloque Único en Formato .md Puro (Copia Completa en 1 Clic)

Usa el botón **"Copiar código"** en la esquina superior derecha del siguiente recuadro para copiar el archivo docs/SANITATION_WORKFLOW_AUDIT.md íntegro sin cortes:

Especificación Oficial: Arquitectura de Saneamiento Guiado, Suficiencia Pedagógica y Evolución Curricular (v2.0)

> **Fecha**: 2026-10-13
> **Estado**: Especificación Oficial Aprobada para Implementación
> **Archivo**: `docs/SANITATION_WORKFLOW_AUDIT.md`
> **Resumen**: Unificación del flujo de saneamiento en la ventana principal (`App.tsx`), ordenamiento topológico por DAG (`DeckDAGService`), motor de "Analizar y Planificar" (`MaterialInspectionCockpit.tsx`), memoria contextual diferida para colisiones (`sanitation_coverage_ledger`), herencia multimodal inteligente (1–8 imágenes), auditoría de suficiencia para principiantes, reto socrático por nodo y Plan de Estudios dinámico multi-etapa.

---

1. Hallazgos Críticos de la Auditoría de Código (Defectos a Corregir)

1. **H-01 (Cola de Saneamiento sin Orden Topológico en `sanitation_service.py`)**:
   - `SanitationService.get_sanitation_queue` ordena las notas únicamente por `_sort_priority` (`bad` -> `preevaluated` -> `unreviewed`) e ignora `DeckDAGService.get_topological_card_order(deck_name)` y el `depth_level` de los nodos.
2. **H-02 ("Sanear Clúster" se salta "Analizar y Planificar" en `sanitation_service.py` y `ClusterSanitationModal.tsx`)**:
   - `generate_note_sanitation_candidates` llama directo a `execute_from_text` / `execute_from_media` sin ejecutar `InspectMaterialUseCase` (`/cards/inspect`) ni `DeckIntelligenceService.match_study_plan` (`/anki/decks/match-study-plan`).
3. **H-03 (Ruptura de Vínculos del DAG en `replace_note_transactionally` dentro de `sanitation_service.py`)**:
   - Al reemplazar una nota monolítica, el servicio borra `source_note_id` de `card_concept_mappings`, pero nunca inserta los nuevos `created_note_ids` en `card_concept_mappings` ni computa sus embeddings vectoriales antes de guardar en `indexed_notes`. Esto provoca que los nodos del DAG pierdan sus tarjetas mapeadas (`card_count = 0`) y caigan erróneamente a estado `UNMAPPED` (`is_orphan = True`).
4. **H-04 (Asunción Negativa sobre Notas de Mala Calidad en `negative_boundaries`)**:
   - `get_cluster_sanitation_preview`, `generate_note_sanitation_candidates` y `GenerateCardsUseCase._apply_mutual_negative_boundaries` inyectan descriptores de notas vecinas como `"PROHIBIDO evaluar conceptos vecinos"`, incluso cuando esas notas vecinas tienen `quality_status == "bad"` o `"unreviewed"`.
5. **H-05 (Herencia Ciega de Imágenes en `App.tsx` y `models.py`)**:
   - `Flashcard` solo soporta un string singular `source_image`, y `App.tsx` (`handleConfirmGenerateFromInspection`) reparte las imágenes con `mediaToInherit.shift()!` secuencialmente sin importar el subtema de cada tarjeta y amontona las sobrantes en `generatedCards[0]`.
6. **H-06 (Eliminación Prematura de Notas Colisionadas en `MaterialInspectionCockpit.tsx`)**:
   - `handleApproveInspection` agrega automáticamente los `matched_note_id` con `status === 'cognitive_debt'` a `obsoleteIds`, eliminando otras tarjetas del mazo sin darles su propio turno de procesamiento.
7. **H-07 (Fragmentación de Modales y Estado Huérfano en `App.tsx` y `DeckStudioModal.tsx`)**:
   - `isSanitationMode` en `App.tsx` depende de `sessionStorage` sin botón de activación real; coexisten `sanitationQueue` y `curationQueue`; `ClusterSanitationModal.tsx` y `RefactorNoteModal.tsx` duplican flujos fuera de la ventana principal.
8. **H-08 (Falta de Validación de Suficiencia para Principiante y Reto Socrático al Cerrar un Nodo del DAG)**:
   - `DeckDAGService.get_deck_dag` marca un nodo como `SANITIZED` solo con verificar que sus tarjetas actuales sean `optimal`, sin comprobar si cubren el 100% de los conceptos que necesita un principiante ni activar un reto interactivo con `SocraticRedTeamingService`.
9. **H-09 (Contrato de Plan de Estudios Estático en `models.py` y `deck_interview_service.py`)**:
   - `LearningPlanContract` carece de soporte para etapas evolutivas (`CurriculumStage`: Etapa 1 Fundamentos 0 a 100 -> Etapa 2 Cloud AWS/Cisco/OCI).

---

2. Invariantes y Decisiones Oficiales de Arquitectura

1. **Unificación en la Ventana Principal (`App.tsx`)**:
   - Se eliminan `ClusterSanitationModal.tsx` y `RefactorNoteModal.tsx`.
   - `AnkiCardPickerModal.tsx` se mantiene únicamente como selector manual para cargar una tarjeta puntual dentro del flujo de *"Analizar y Planificar"* en la ventana principal.

- La ventana principal muestra la **Cola Topológica Unificada de Saneamiento** (ordenada de raíces `depth_level = 0` a hojas avanzadas) e invoca siempre *"Analizar y Planificar"* (`MaterialInspectionCockpit.tsx`).
2. **Política de Colisiones con Tarjetas de Mala Calidad (Opción B — Turno Propio con Memoria Contextual)**:
- Si la nota actual A colisiona en un subtema con una nota existente B de mala calidad (`bad` o `unreviewed`), el sistema notifica la deuda de B, mantiene activo el subtema en el turno de A, genera las tarjetas atómicas con su justificación pedagógica, **no borra B en el turno de A**, y registra el subtema cubierto en la tabla `sanitation_coverage_ledger`.
- Cuando la cola llega al turno propio de la nota B, el sistema lee `sanitation_coverage_ledger`, marca como `DOMINATED` los ejes ya cubiertos por las tarjetas creadas en el turno de A, y solo genera tarjetas para los conceptos restantes de B.
3. **Eliminación Total de la Nota Monolítica Original**:
- Al confirmar la sustitución transaccional (`replace_note_transactionally`), se insertan las nuevas tarjetas atómicas en Anki y se elimina definitivamente la nota monolítica original (`delete_notes`).
4. **Herencia Multimodal Inteligente por Eje Temático (1 a 8 Imágenes)**:
- `InspectMaterialUseCase` agrupa las imágenes (1 a 8) de la nota original por `CoreTheme` (`associated_diagrams` + `diagram_roles`).
- Cada `Flashcard` generada recibe en `source_images: List[str]` el subconjunto exacto de imágenes relevantes para su eje temático.
5. **Cierre de Nodo del DAG: Suficiencia para Principiante + Reto Socrático Interactivo**:
- Al alcanzar un nodo del DAG el 100% de sus tarjetas mapeadas en estado `optimal`, se dispara:
a) **Auditoría de Suficiencia del Nodo**: Verifica si las tarjetas actuales bastan para que un principiante entienda el tema sin brechas; si hay brechas (`orphan_gaps`), permite generarlas en el acto.
b) **Reto Socrático Interactivo**: Con 0 brechas, `SocraticRedTeamingService` lanza un reto interactivo sobre el nodo; al superarlo, el nodo pasa al estado `MASTERED`.
6. **Plan de Estudios Dinámico Multi-Etapa (Fase 2)**:
- `LearningPlanContract` modela una lista de `CurriculumStage` (Etapa 1: Fundamentos de 0 a 100 -> Etapa 2: Especialización Cloud / Certificación), adaptando el rigor de atomización al nivel del estudiante (`beginner`, `intermediate`, `advanced`).

---

3. Flujo de Datos y Estados del Nodo en el DAG

- **Estados del Nodo (`ConceptNode.status`)**:
1. `UNMAPPED`: 0 tarjetas vinculadas en el mazo (brecha huérfana).
2. `RAW`: Tiene tarjetas vinculadas, pero al menos una está en estado `bad` o `unreviewed`.
3. `SANITIZED`: 100% de sus tarjetas vinculadas están en estado `optimal`, pendiente de auditoría de suficiencia para principiante.
4. `SUFFICIENT`: Auditoría de cobertura temática certificada al 100% (0 brechas para el nivel del usuario), listo para el Reto Socrático.
5. `MASTERED`: Reto Socrático superado; desbloquea de forma segura los nodos dependientes y la expansión curricular.

---

4. Plan de Implementación por Bloques

Bloque 1: Backend Core, Modelos y Base de Datos SQLite
- **`backend/app/domain/models.py`**:
- Agregar `source_images: List[str]` en `Flashcard` y `diagram_roles: Dict[str, str]` en `CoreTheme`.

  - Agregar `LearnerExpertiseLevel`, `CurriculumStage` y lista `stages: List[CurriculumStage]` en `LearningPlanContract`.
  - Agregar modelos de `SanitationCoverageLedgerEntry`, `NodeMasteryStatus` y campos de progreso topológico en `SanitationQueueResponse` y `TransactionalReplaceResponse`.
- **`backend/app/adapters/db/deck_knowledge_store.py`**:
  - Crear tabla `sanitation_coverage_ledger` y métodos CRUD (`record_ledger_coverage`, `get_ledger_entries_for_note`, `delete_ledger_entries_for_note`).
- **`backend/app/services/sanitation_service.py`**:
  - Reescribir `get_sanitation_queue` con ordenamiento topológico del DAG (`depth_level` ASC + `topological_order`).
  - Reparar `replace_note_transactionally` para calcular embeddings de las nuevas tarjetas, transferir los vínculos en `card_concept_mappings`, registrar entradas en `sanitation_coverage_ledger` y detectar si un nodo del DAG acaba de completarse.

Bloque 2: Inspección Multimodal (1–8 Imágenes), Cruce con Ledger y Reto Socrático
- **`backend/app/services/inspect_material_service.py` y `gemini_card_generator_adapter.py`**:
  - Inyectar nivel de experiencia de la etapa activa del contrato (`beginner` / `intermediate` / `advanced`).
  - Etiquetar las imágenes 1..8 con sus nombres de archivo y agruparlas por `CoreTheme` en `associated_diagrams`.
- **`backend/app/services/deck_intelligence_service.py`**:
  - Enriquecer `match_study_plan` cruzando contra `sanitation_coverage_ledger` y asegurando que las colisiones con notas de mala calidad (`COGNITIVE_DEBT`) no borren la nota vecina ni bloqueen la generación en el turno actual.
- **`backend/app/services/socratic_red_teaming_service.py` y `backend/app/api/routes.py`**:
  - Añadir flujo interactivo de certificación socrática por nodo del DAG (`start_node_socratic_challenge` y `evaluate_node_socratic_response`).

Bloque 3: Unificación en Frontend y Limpieza de Modales Obsoletos
- **Eliminar**: `frontend/src/components/ClusterSanitationModal.tsx` y `frontend/src/components/RefactorNoteModal.tsx`.
- **`frontend/src/App.tsx` y `frontend/src/components/CurationQueueBanner.tsx`**:
  - Unificar `sanitationQueue` y `curationQueue` en el panel principal de **Cola Topológica de Saneamiento**.
  - Reemplazar el reparto `.shift()` de imágenes por el renderizado exacto de `card.source_images`.
- **`frontend/src/components/MaterialInspectionCockpit.tsx`**:
  - Impedir que las notas colisionadas con deuda cognitiva se agreguen a `obsoleteIds` en el turno actual; mostrar el estado del `sanitation_coverage_ledger` y el selector visual de imágenes por `CoreTheme`.
- **`frontend/src/components/DeckStudioModal.tsx`**:
  - Conectar el inicio de saneamiento y selección de notas con deuda directamente a la cola topológica de la ventana principal (`App.tsx`).